

Introduction to the Xilinx Vivado and ZedBoard Platform

2026 / 03 / 27

李紹齊

Outline

- RTL Synthesis and Implementation via Xilinx Vivado
 - Complete the Verilog code to drive the 8 LEDs on the ZedBoard in a loop
- In-Class Practice
- Software Development via Xilinx SDK
 - Run Llama2.c on the ZedBoard
- In-Class Practice



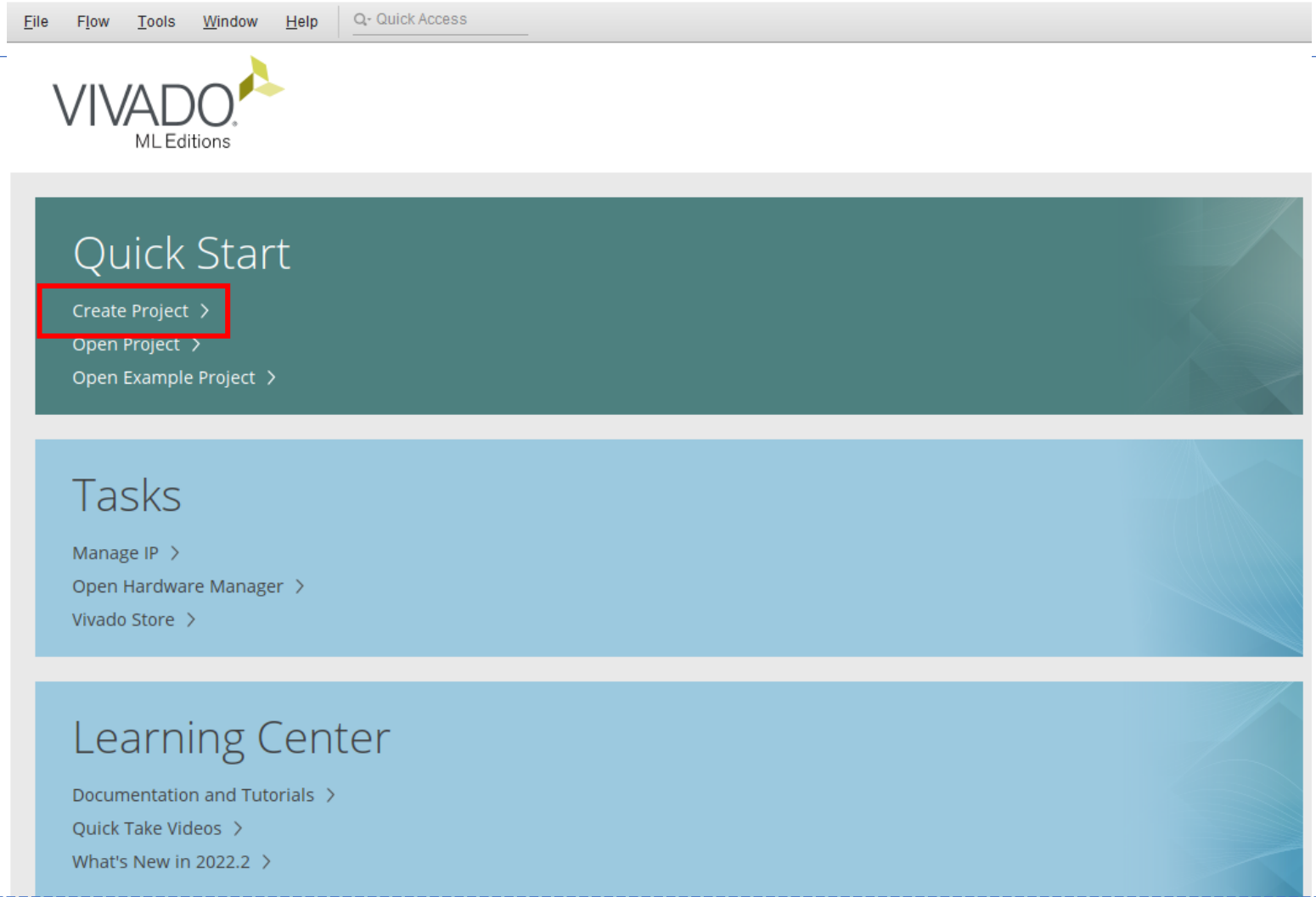
Outline

- RTL Synthesis and Implementation via Xilinx Vivado
 - Create Project
 - Create RTL
 - Create Block Design
 - Create Constraint File and Assign PINs
 - Synthesis and Implementation
 - Program to FPGA
- In-Class Practice
- Software Development via Xilinx SDK
- In-Class Practice



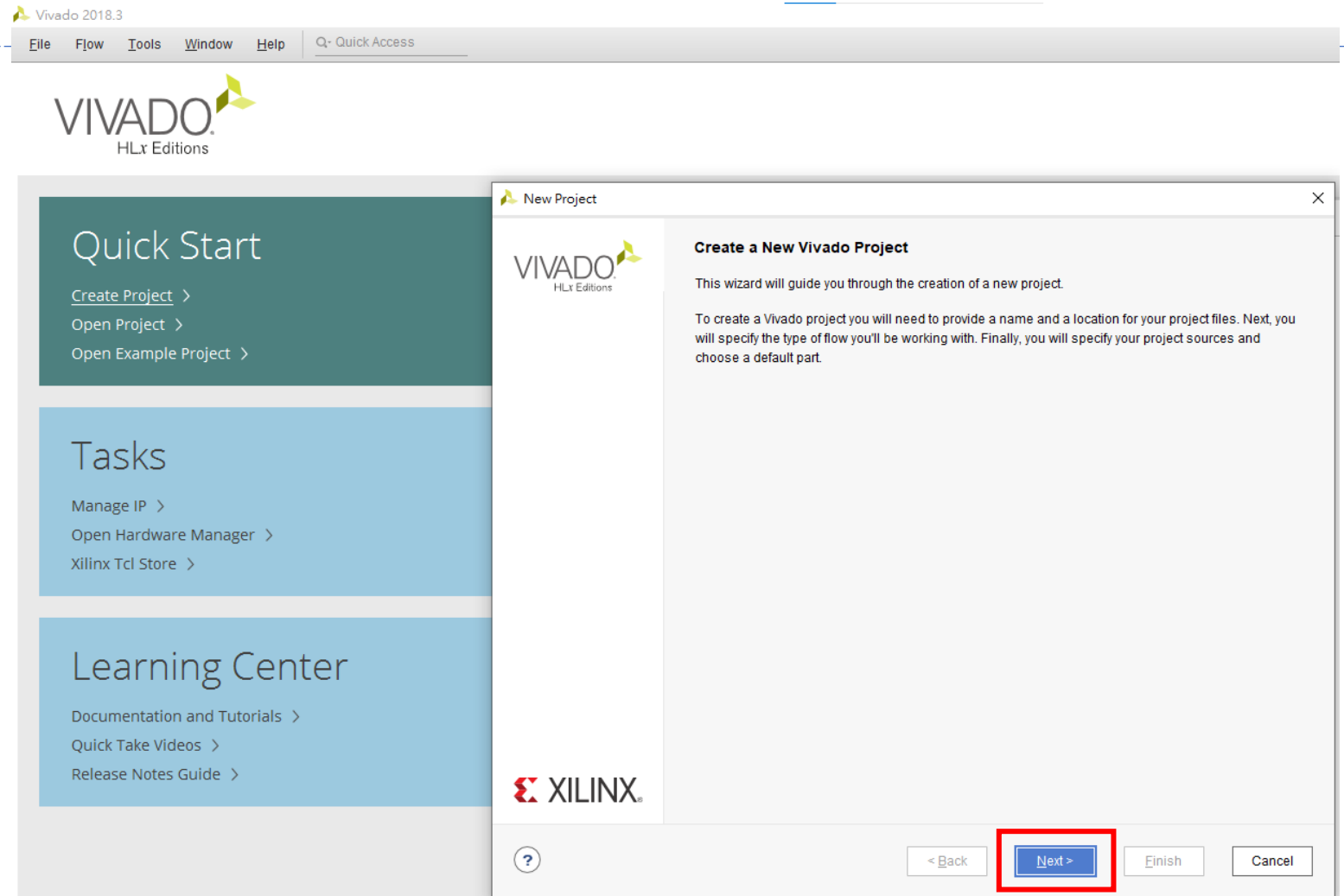
Create Project

1. File → Project → New
or Create Project



Create Project

2. Click “Next”



Create Project

3. Fill Project Name and
Select Project Location
→ Next

(You can create a folder to store your own project)

Remember not to include any
Chinese word in your path

New Project

Project Name

Enter a name for your project and specify a directory where the project data files will be stored.

1.

Project name: Lab1_Led

Project location: D:/SOPC_Lab1

☒ Create project subdirectory

Project will be created at: D:/SOPC_Lab1/Lab1_Led

2.

< Back Next > Finish Cancel

Create Project

4. RTL Project → Next
(If you want to add
your source first ,
you can click that
line)

New Project

Project Type
Specify the type of project to create.

1.

☒ **_RTL Project**
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
☒ Do not specify sources at this time

☐ **_Post-synthesis Project:** You will be able to add sources, view device resources, run design analysis, planning and implementation.
☐ Do not specify sources at this time

☐ **_I/O Planning Project**
Do not specify design sources. You will be able to view part/package resources.

☐ **_Imported Project**
Create a Vivado project from a Synplify, XST or ISE Project File.

☐ **_Example Project**
Create a new Vivado project from a predefined template.

2.

? < Back **Next >** Finish Cancel

Create Project

5. Boards

→ Choose “ZedBoard”

→ Next

New Project

Default Part

Choose a default Xilinx part or board for your project.

1.

Part: **Boards**

[Reset All Filters](#)

Vendor: All Name: All

Search: Q

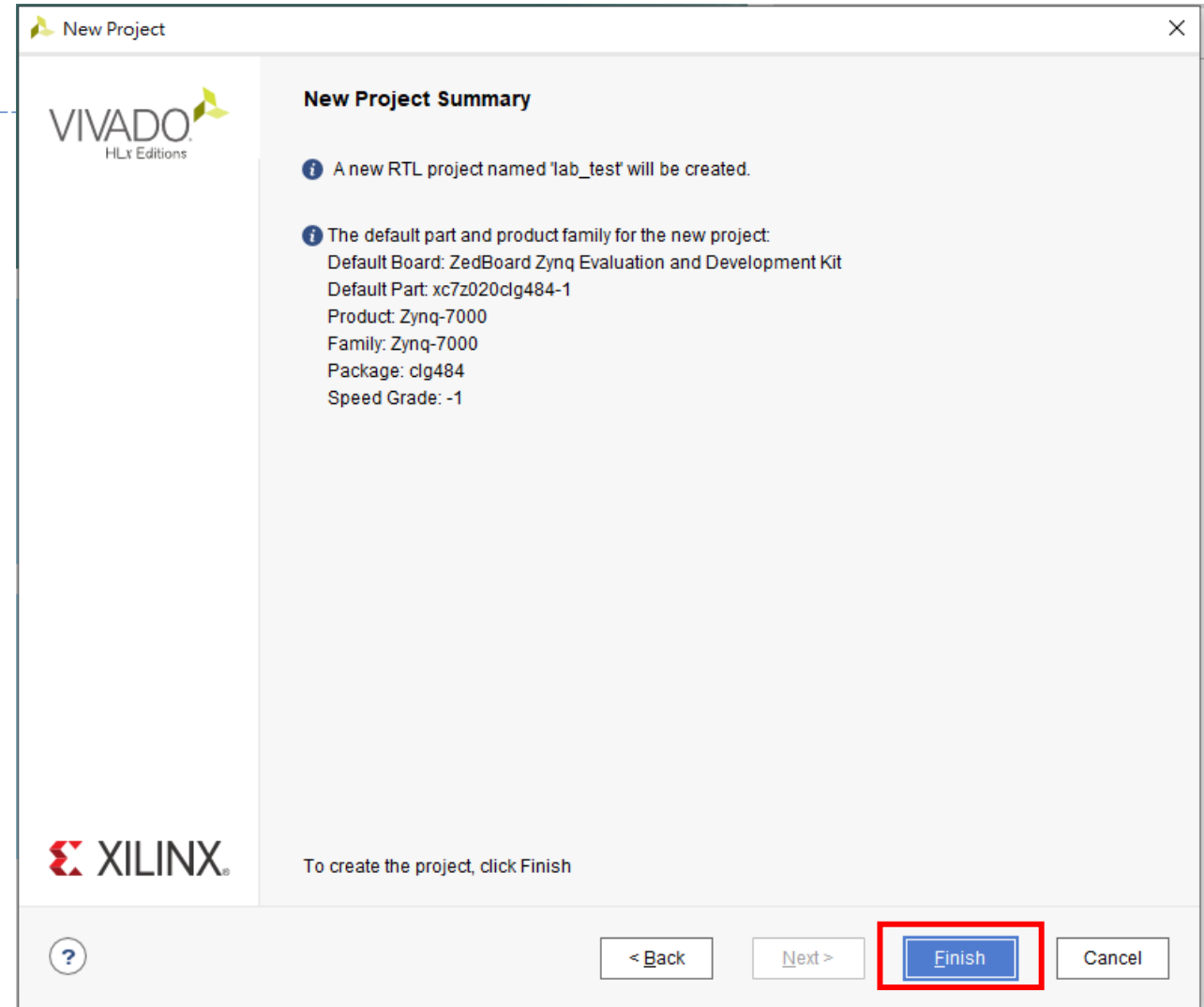
Display Name	Preview	Vendor	File Version	Part
Alpha-Data ADM-PCIE-7V3		alpha-data.com	1.1	xc7vx690tffg1157
2. Kintex-Ultrascale Alphadata board		alpha-data.com	1.0	xcku060-ffva1156
ZedBoard Zynq Evaluation and Development Kit		em.avnet.com	1.4	xc7z020clg484-1
Artix-7 AC701 Evaluation Platform		xilinx.com	1.4	xc7a200tffg676-2
Alveo U200 Data Center Accelerator Card		xilinx.com	1.0	Acceleration Platf
Alveo U250 Data Center Accelerator Card		xilinx.com	1.0	Acceleration Platf

3.

[? Back](#) [Next >](#) [Finish](#) [Cancel](#)

Create Project

6. Click “ Finish ”



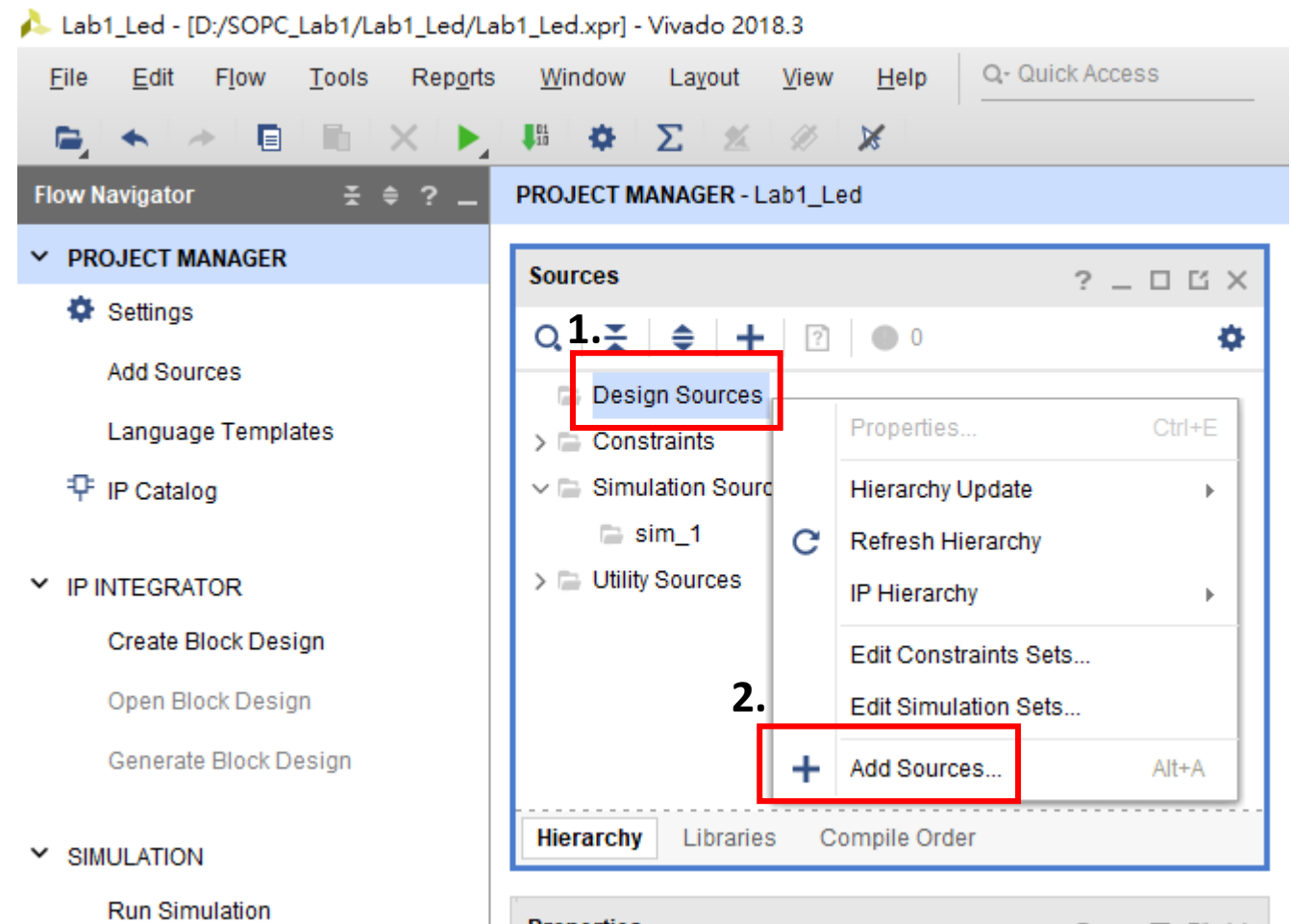
Outline

- RTL Synthesis and Implementation via Xilinx Vivado
 - Create Project
 - Create RTL
 - Create Block Design
 - Create Constraint File and Assign PINs
 - Synthesis and Implementation
 - Program to FPGA
- In-Class Practice
- Software Development via Xilinx SDK
- In-Class Practice



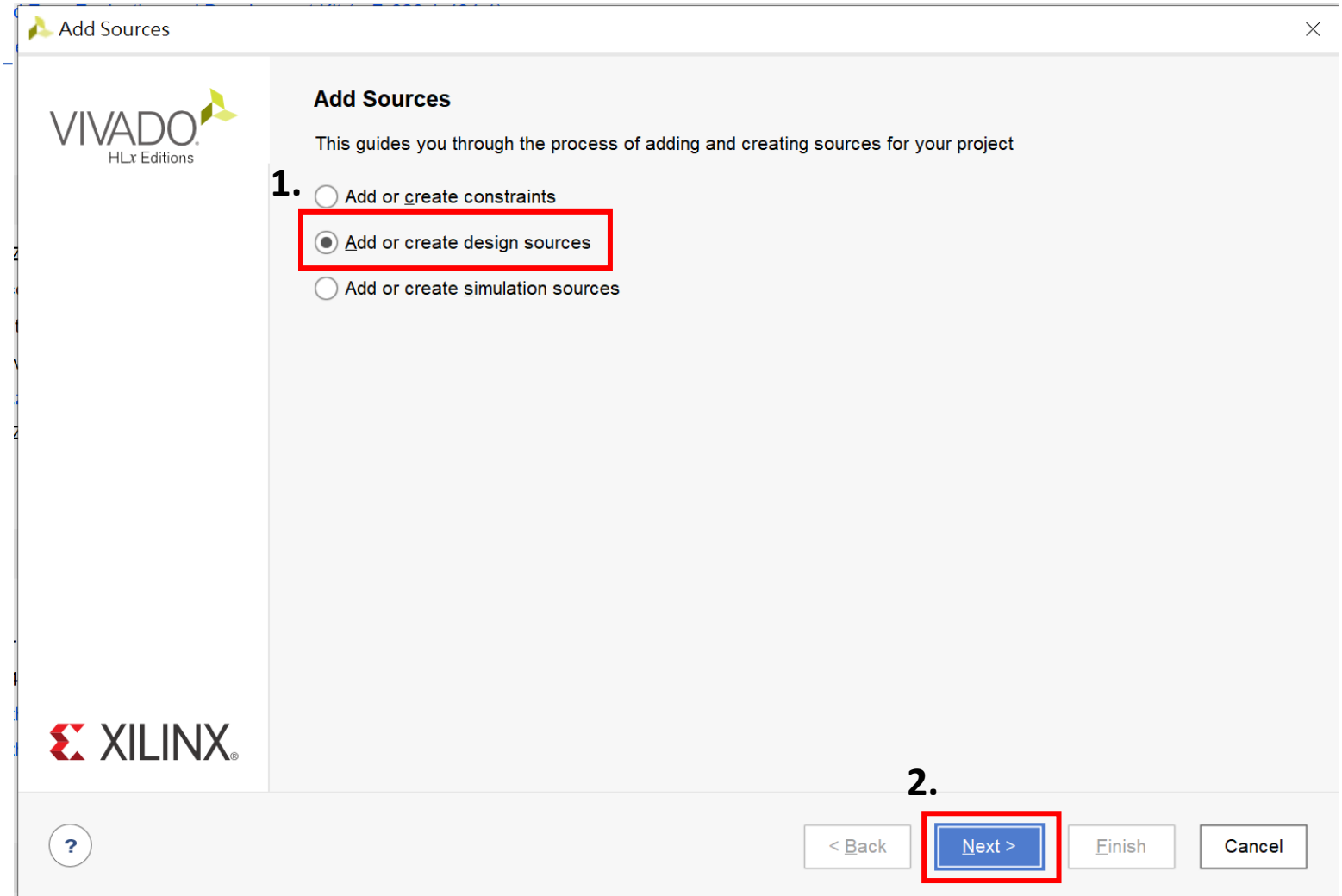
Create RTL Code

1. Right Click “ Design Source ”
→ Add Source



Create RTL Code

2. Click “ Add or Create Design Source ”
→ Click “ Next ”



Create RTL Code

3. Click “Add Files”

→ Choose the RTL code

→ Click “Finish”

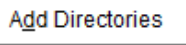
 Add Sources ×

Add or Create Design Sources

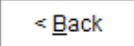
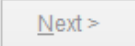
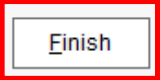
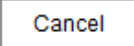
Specify HDL, netlist, Block Design, and IP files, or directories containing those file types to add to your project. Create a new source file on disk and add it to your project.

2.    

	Index	Name	Library	Location
	1	LED.v	xil_defaultlib	G:/lab1

1.   

☐ Scan and add RTL include files into project
☐ Copy sources into project
☒ Add sources from subdirectories

3.     

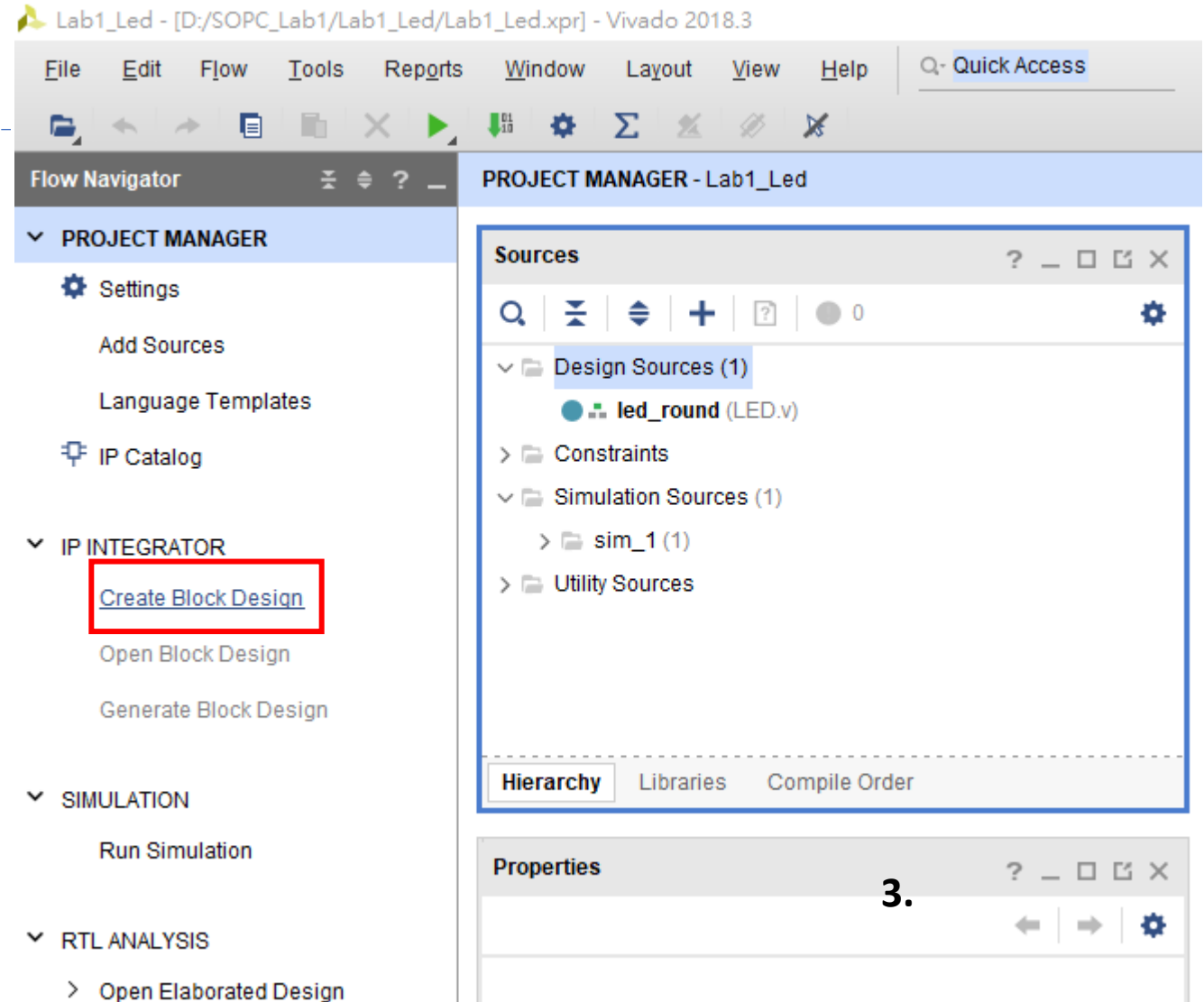
Outline

- RTL Synthesis and Implementation via Xilinx Vivado
 - Create Project
 - Create RTL
 - Create Block Design
 - Create Constraint File and Assign PINs
 - Synthesis and Implementation
 - Program to FPGA
- In-Class Practice
- Software Development via Xilinx SDK
- In-Class Practice



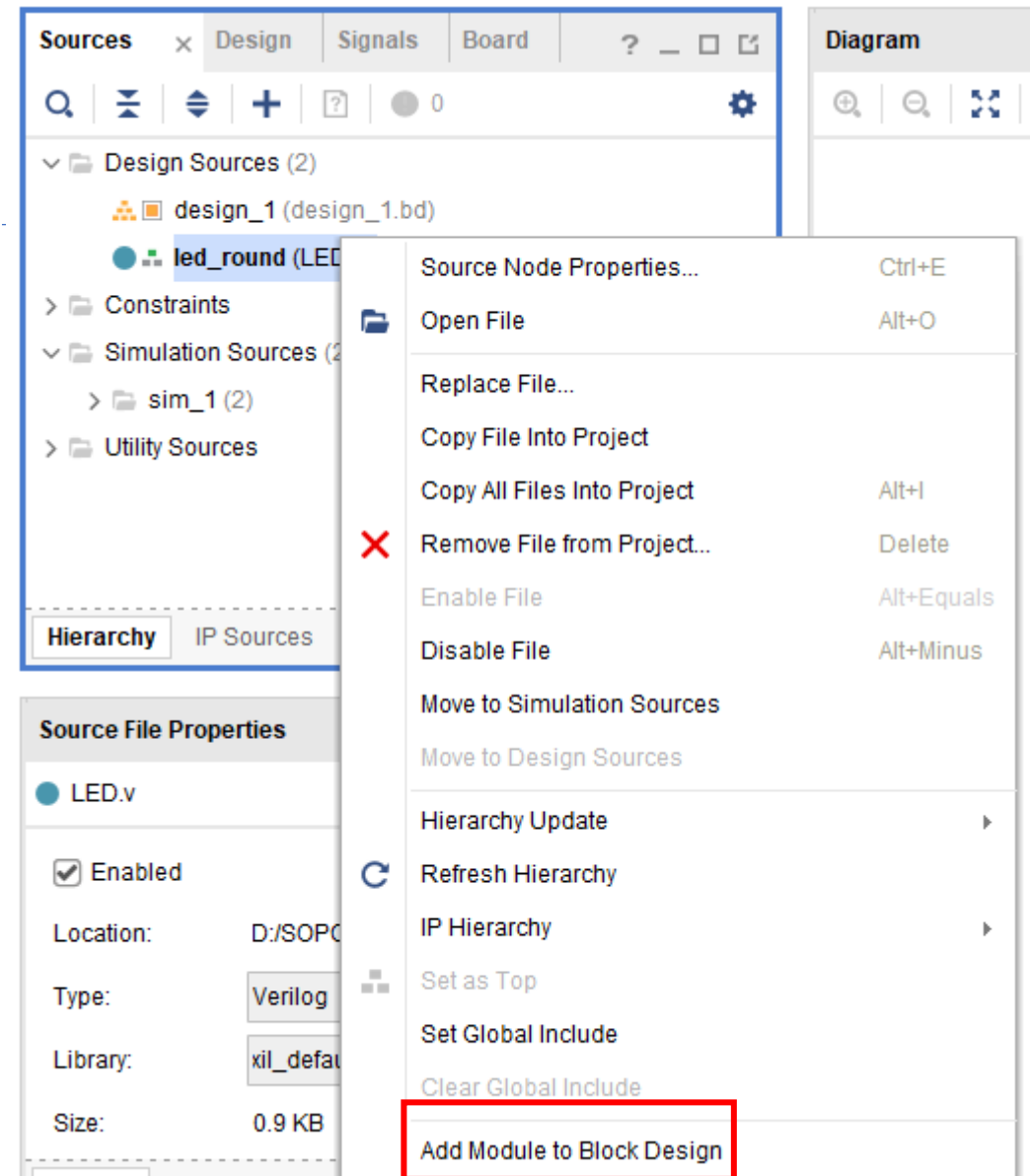
Create Block Design

1. Click “Create Block Design”



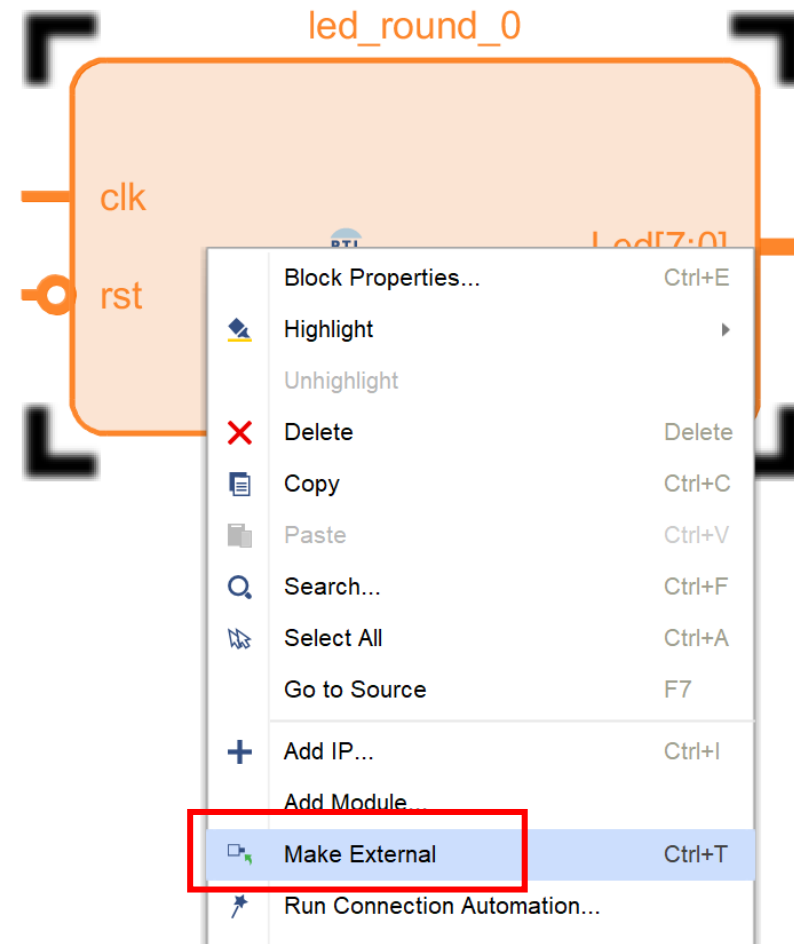
Create Block Design

2. Right Click your module
→ Add Module to Block Design



Create Block Design

3. Right Click the Instance
→ Make External



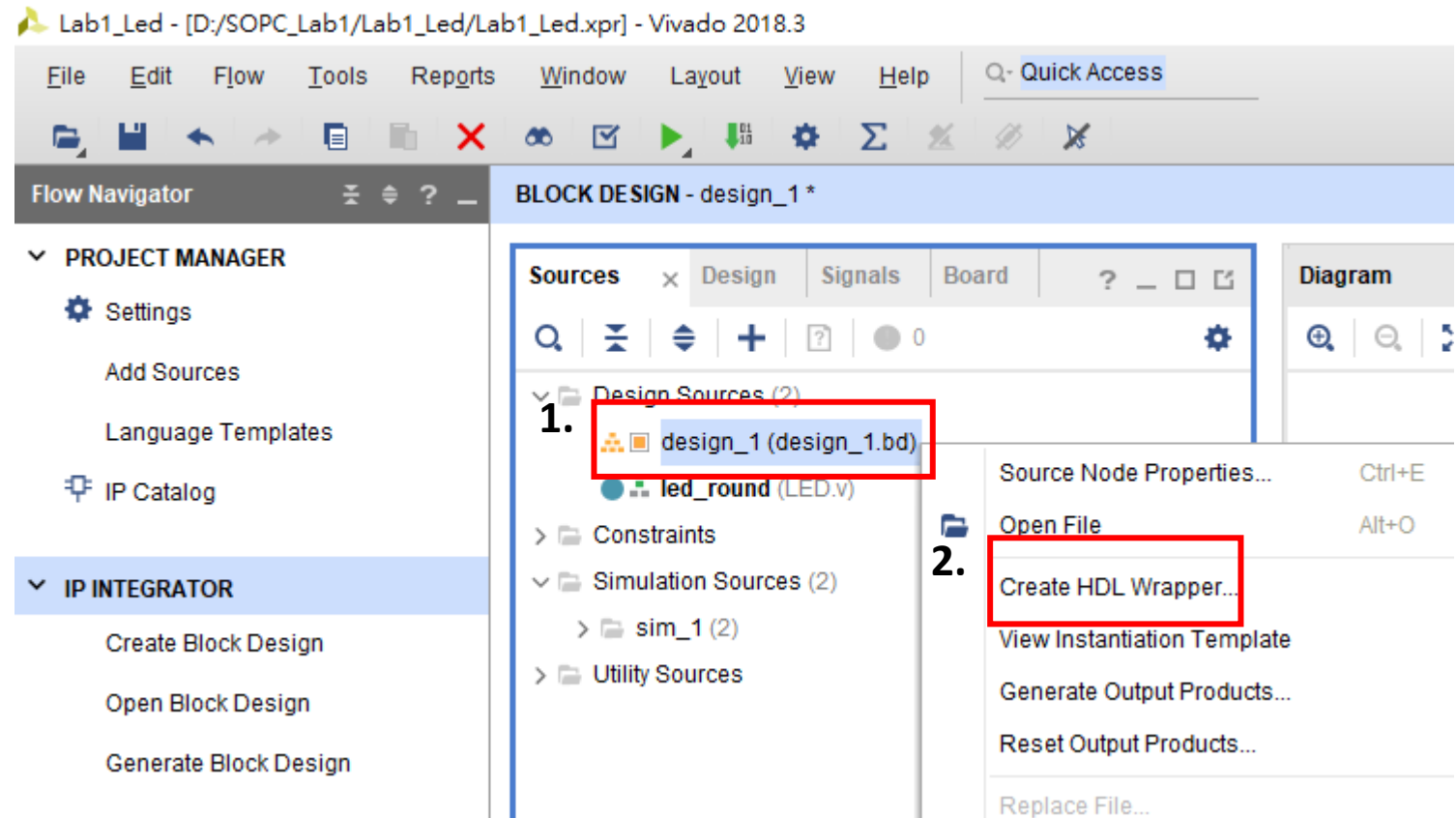
Create Block Design

4. Check the PINs are generated



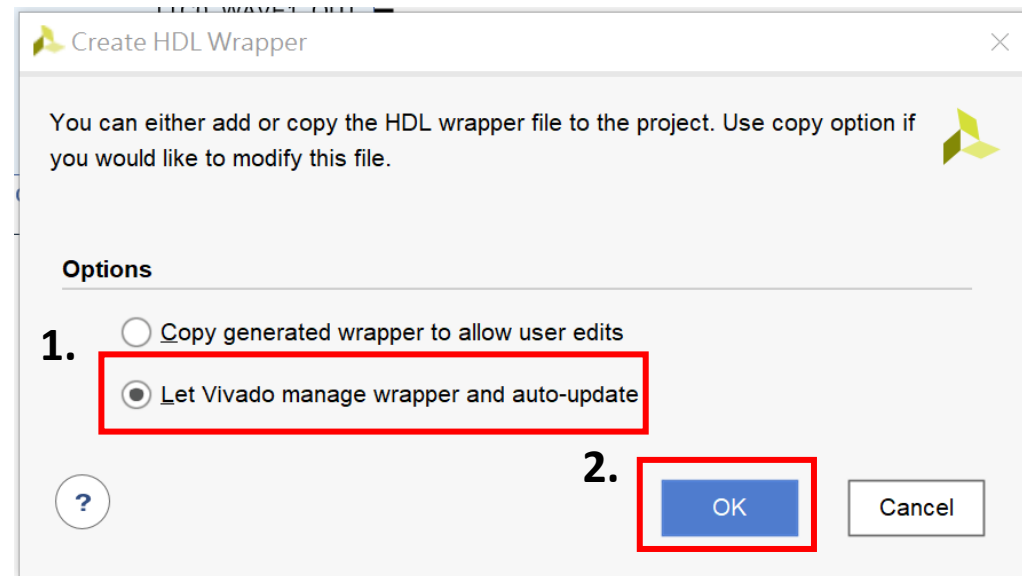
Create Block Design

5. Right Click Block Design
→ Create HDL Wrapper



Create Block Design

6. Let Vivado manage wrapper and auto-update
→ Click “ OK ”



Outline

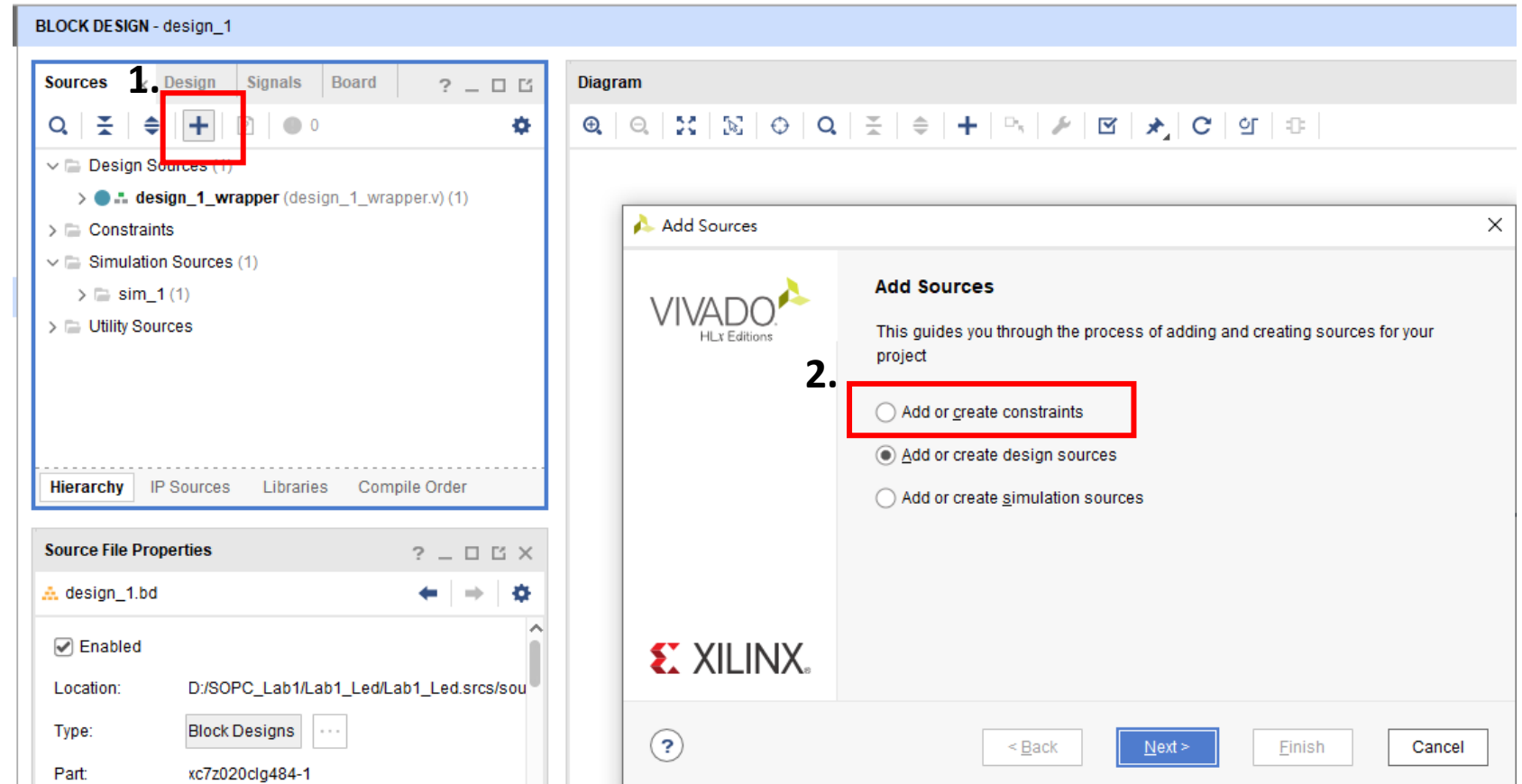
- RTL Synthesis and Implementation via Xilinx Vivado
 - Create Project
 - Create RTL
 - Create Block Design
 - Create Constraint File and Assign PINs
 - Synthesis and Implementation
 - Program to FPGA
- In-Class Practice
- Software Development via Xilinx SDK
- In-Class Practice



Create Constraint File and assign PINs

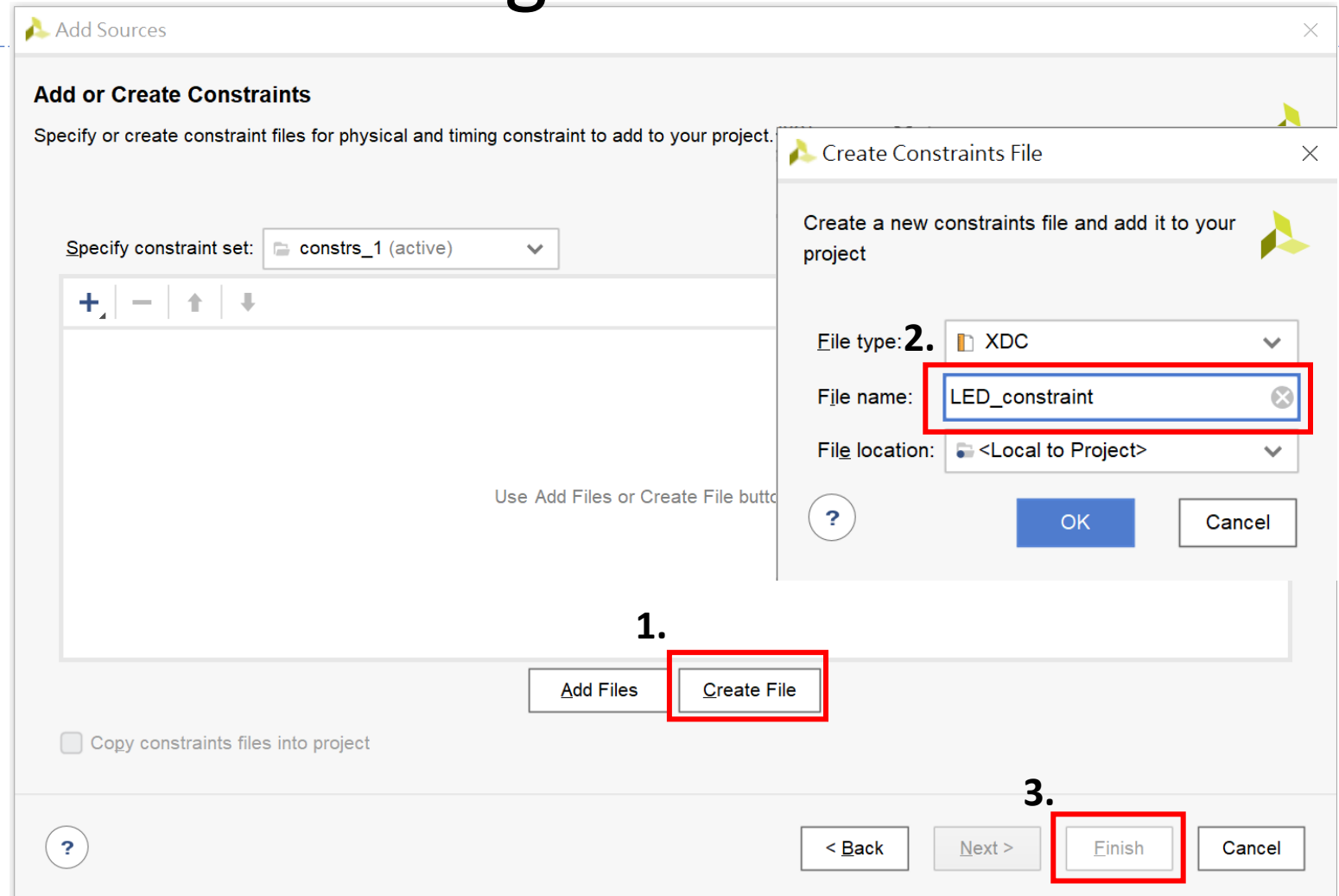
1. Click “ + ”

→ Click “ Add or Create Constraints”



Create Constraint File and Assign PINs

2. Click “ Create Files ”
 - Fill the File name
 - Click “ Finish ”



Create Constraint File and Assign PINs

3. Click Constraint file and write the constraint file

```
# Led0
set_property PACKAGE_PIN T22 [get_ports {Led_0[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Led_0[0]}]

# Led1
set_property PACKAGE_PIN T21 [get_ports {Led_0[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Led_0[1]}]
```

Map the module PIN to LED

```
# clk(GCLK)
set_property PACKAGE_PIN Y9 [get_ports {clk_0}]
set_property IOSTANDARD LVCMOS33 [get_ports {clk_0}]

# rst(BTNU)
set_property PACKAGE_PIN T18 [get_ports {rst_0}]
set_property IOSTANDARD LVCMOS33 [get_ports {rst_0}]
```

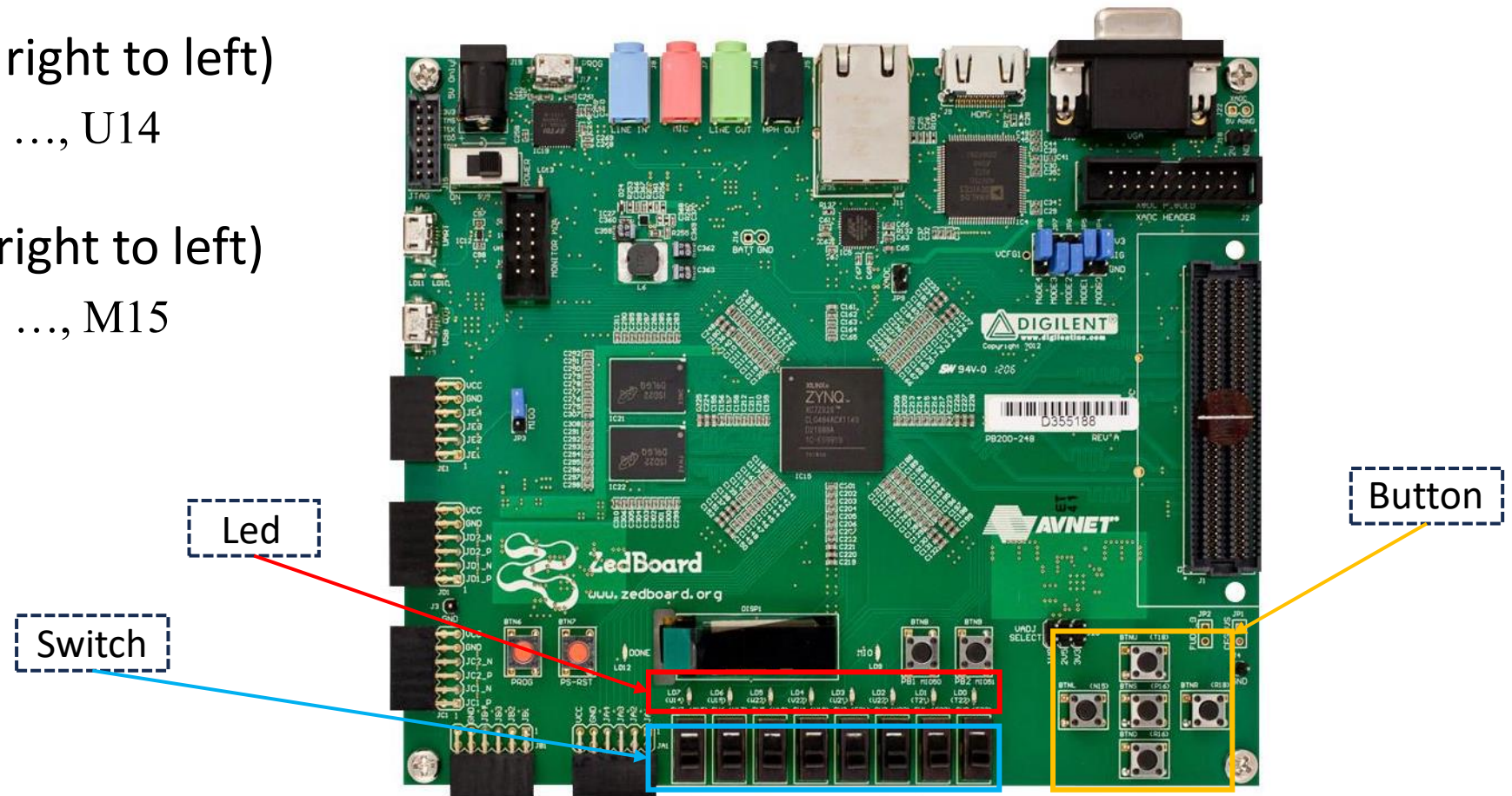
Map clk , rst on ZedBoard

```
#timing
create_clock -period 10.000 -name clk -waveform {0.000 5.000} [get_ports clk_0]
```

Clock information for synthesis

Create Constraint File and Assign PINs

1. Led 0 ~ Led 7 (from right to left)
 - Pin assign : T22, T21, ..., U14
2. SW 0 ~ SW 7 (from right to left)
 - Pin assign : F22, G22, ..., M15



Outline

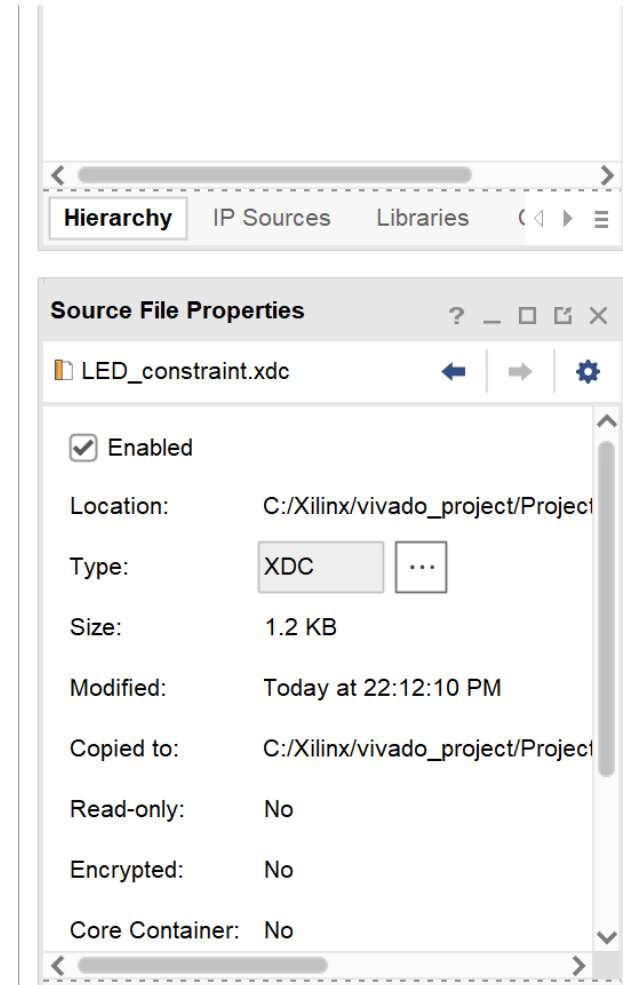
- RTL Synthesis and Implementation via Xilinx Vivado
 - Create Project
 - Create RTL
 - Create Block Design
 - Create Constraint File and Assign PINs
 - Synthesis and Implementation
 - Program to FPGA
 - In-Class Practice
 - Software Development via Xilinx SDK
 - In-Class Practice
-



Synthesis and Implementation

1. Run Synthesis & Implementation

- ▼ RTL ANALYSIS
 - > Open Elaborated Design
- 1. ▼ SYNTHESIS
 - ▶ Run Synthesis
 - > Open Synthesized Design
- 2. ▼ IMPLEMENTATION
 - ▶ Run Implementation
 - > Open Implemented Design
- ▼ PROGRAM AND DEBUG
 - ⬇️ Generate Bitstream
 - > Open Hardware Manager



Synthesis and Implementation

4. See Project Summary

1. timing
2. utilization
3. power

Project Summary

Overview | Dashboard

Connectors: no connections

Repository path: C:/Xilinx/Vivado/2018.3/data/boards/board_files

URL: <http://www.zedboard.org>

Board overview: ZedBoard Zynq Evaluation and Development Kit

[Changes](#)

Synthesis

Status: ✔ Complete

Messages: ⚠ 3 warnings

Active run: [synth_1](#)

Part: xc7z020clg484-1

Strategy: [Vivado Synthesis Defaults](#)

Report Strategy: [Vivado Synthesis Default Reports](#)

Implementation

Status: ✔ Complete

Messages: ⚠ 1 warning

Active run: [impl_1](#)

Part: xc7z020clg484-1

Strategy: [Vivado Implementation Defaults](#)

Report Strategy: [Vivado Implementation Default Reports](#)

Incremental implementation: [None](#)

DRC Violations

Summary: ⚠ 1 warning

[Implemented DRC Report](#)

Timing

Worst Negative Slack (WNS): 5.722 ns

Total Negative Slack (TNS): 0 ns

Number of Failing Endpoints: 0

Total Number of Endpoints: 33

[Implemented Timing Report](#)

Utilization

Post-Synthesis | **Post-Implementation**

Graph | Table

LUT	1%
FF	1%
IO	5%
BUFG	3%

Utilization (%)

Power

Summary | On-Chip

Total On-Chip Power: 0.104 W

Junction Temperature: 26.2 °C

Thermal Margin: 58.8 °C (4.9 W)

Effective θ_{JA} : 11.5 °C/W

Power supplied to off-chip devices: 0 W

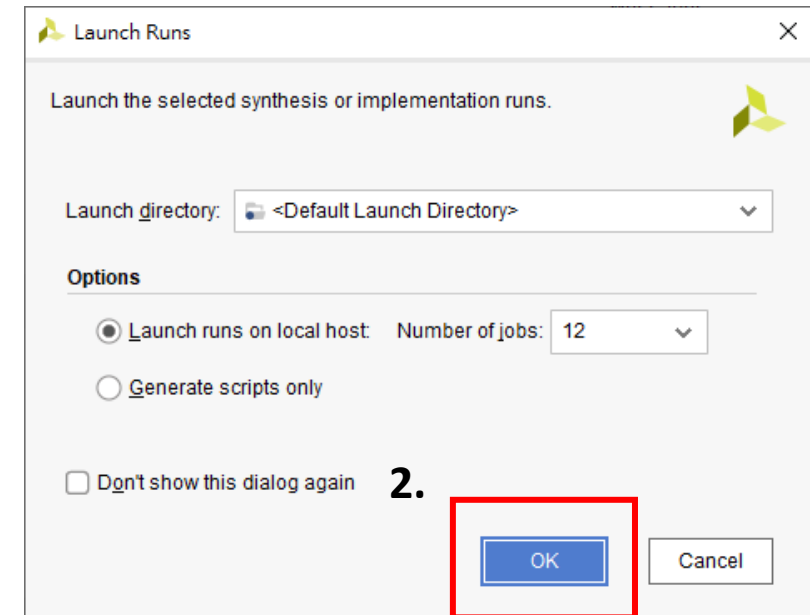
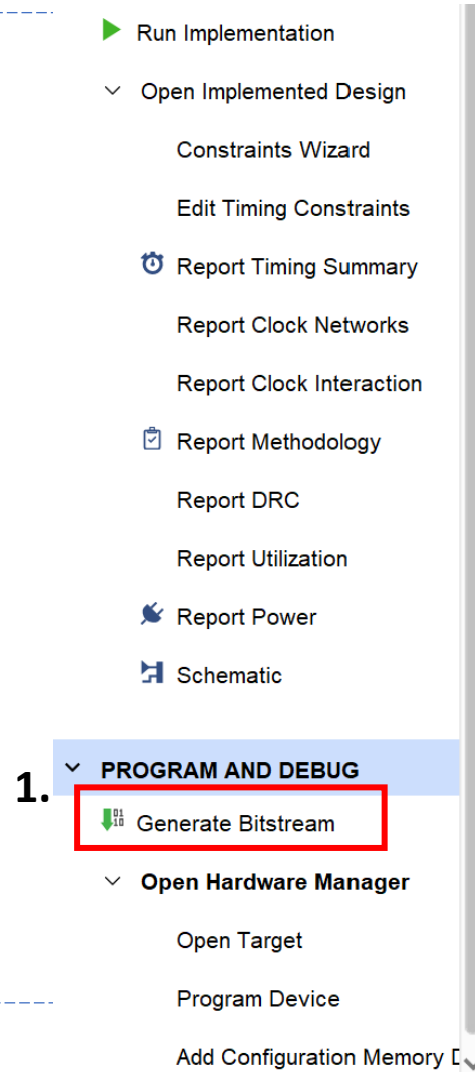
Confidence level: [Low](#)

[Implemented Power Report](#)

Make sure no error !

Synthesis and Implementation

5. Generate Bitstream



Outline

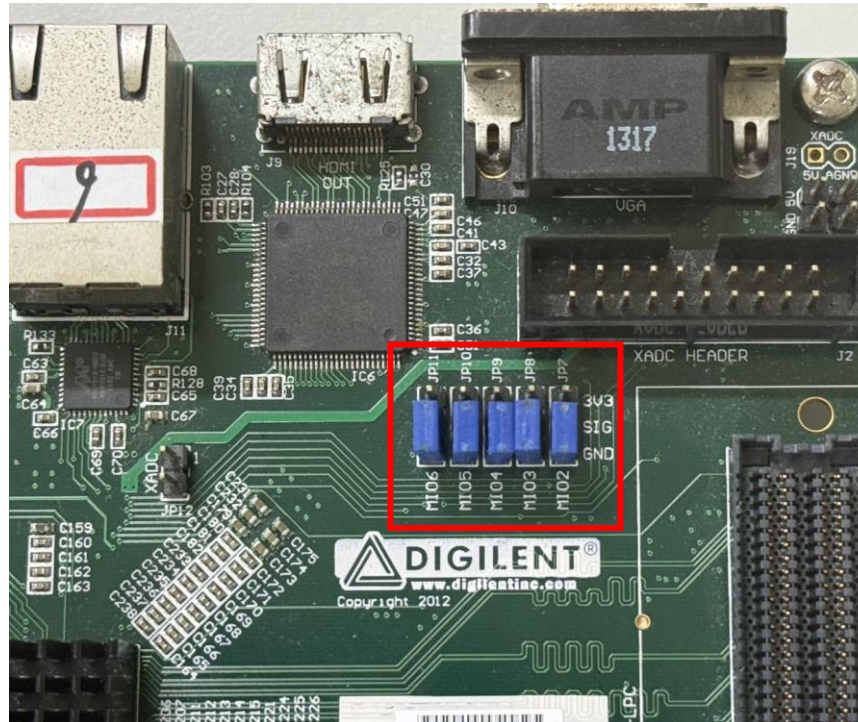
- RTL Synthesis and Implementation via Xilinx Vivado
 - Create Project
 - Create RTL
 - Create Block Design
 - Create Constraint File and Assign PINs
 - Synthesis and Implementation
 - Program to FPGA
 - In-Class Practice
 - Software Development via Xilinx SDK
 - In-Class Practice
-



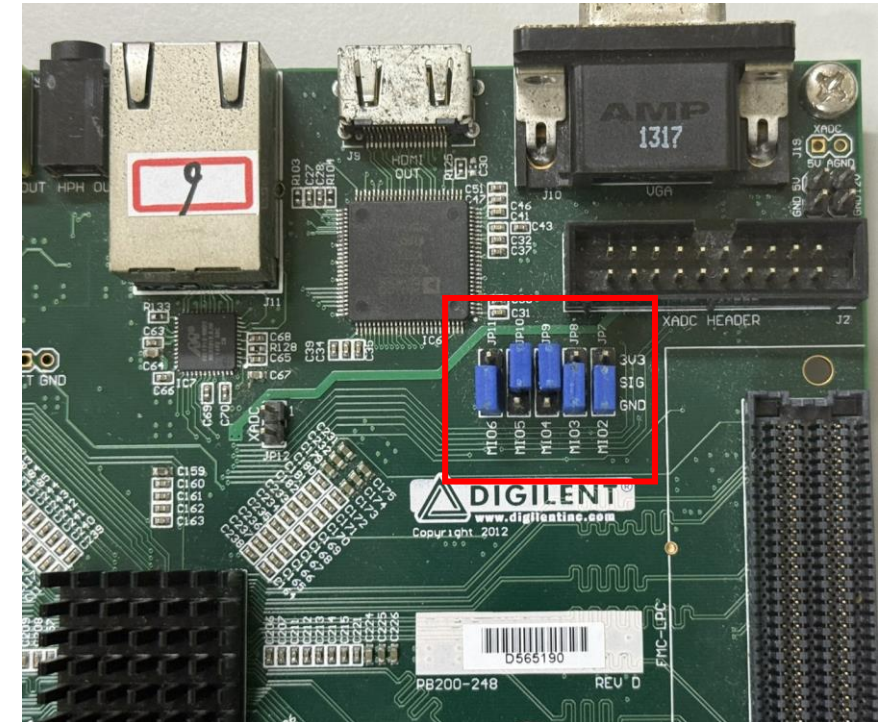
Program to FPGA

1. Configuration

- 1) JTAG
- 2) SD Card



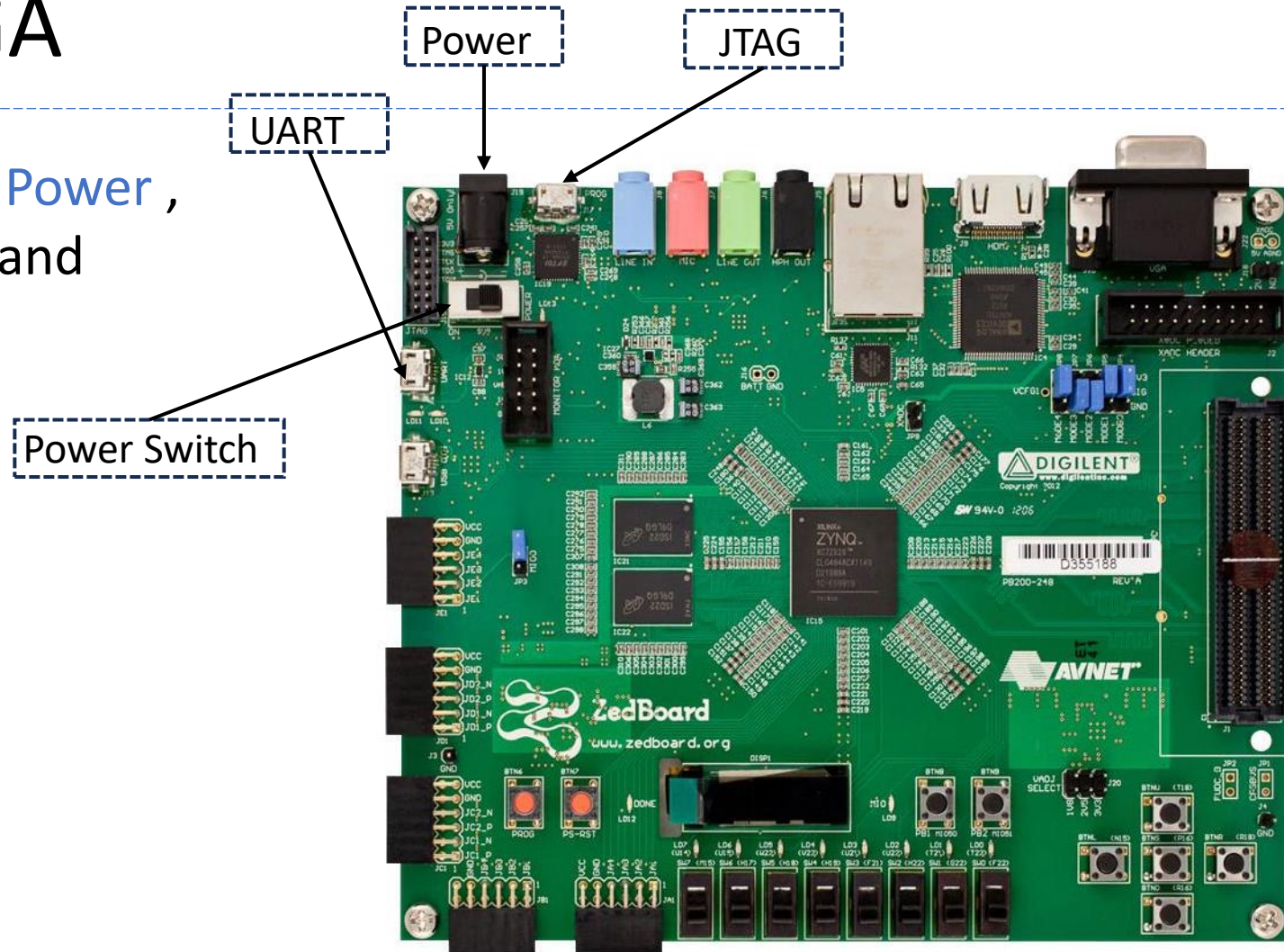
➤ JTAG



➤ SD Card

Program to FPGA

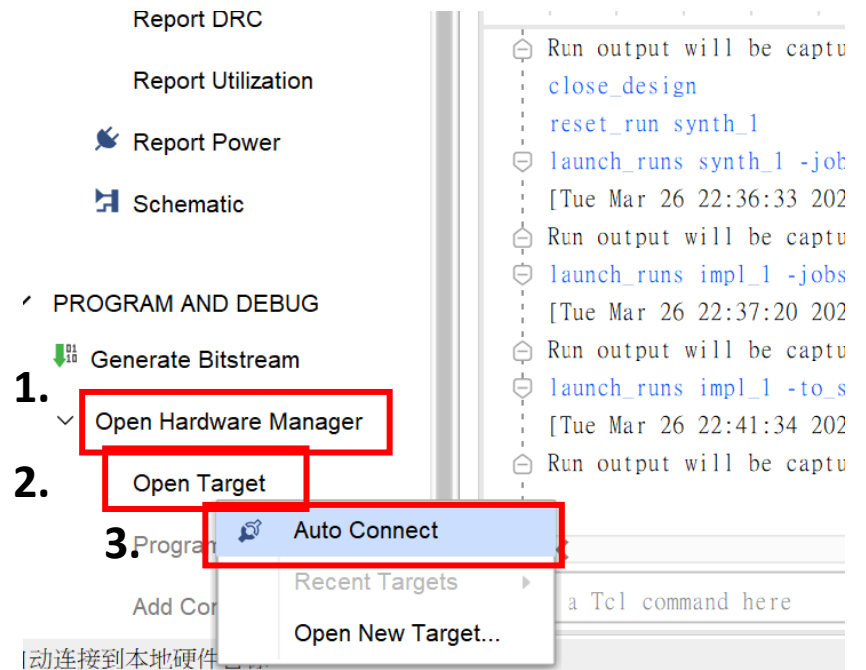
1. Connect ZedBoard with **Power** , **UART** , and **JTAG** cable , and turn on the power



Program to FPGA

2. Open Hardware Manager

- Open Target
- Auto Connect

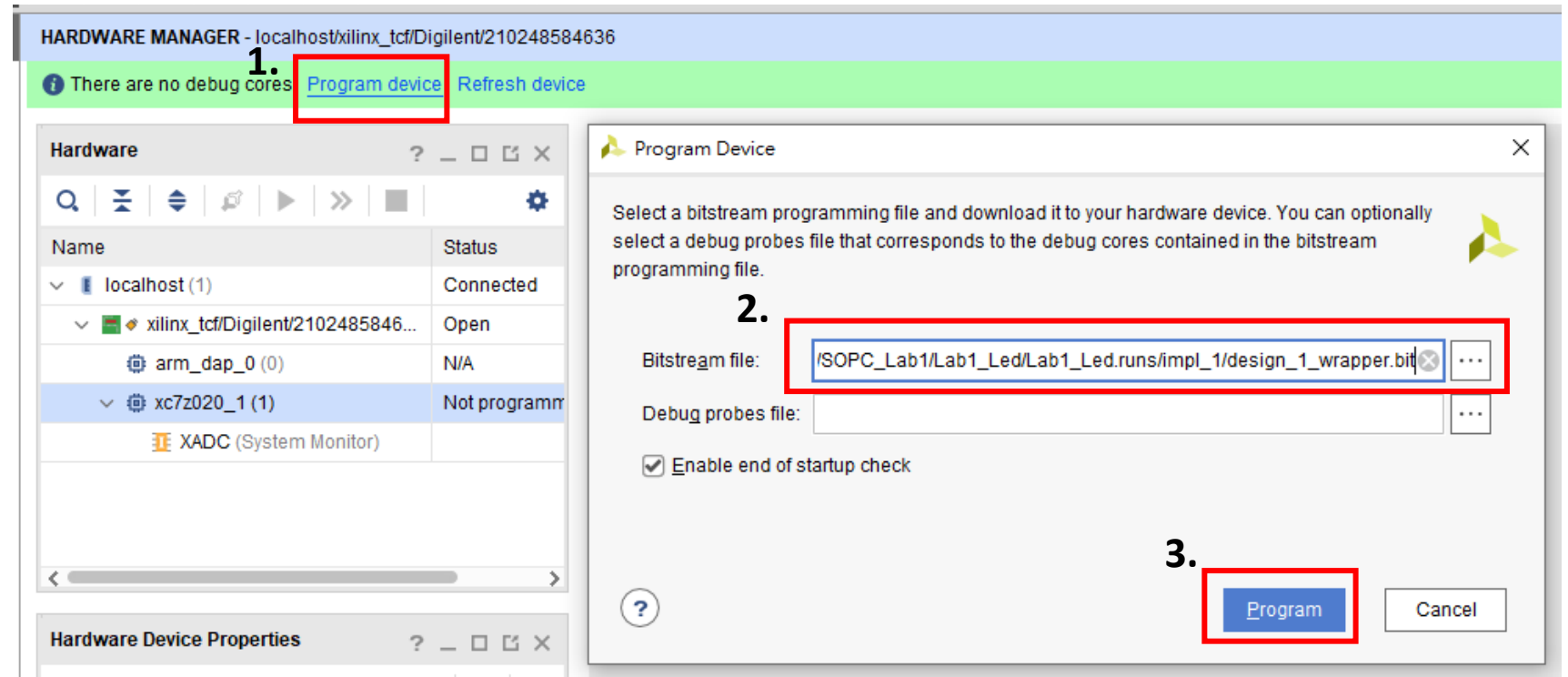


Program to FPGA

3. Click “ Program Device ”

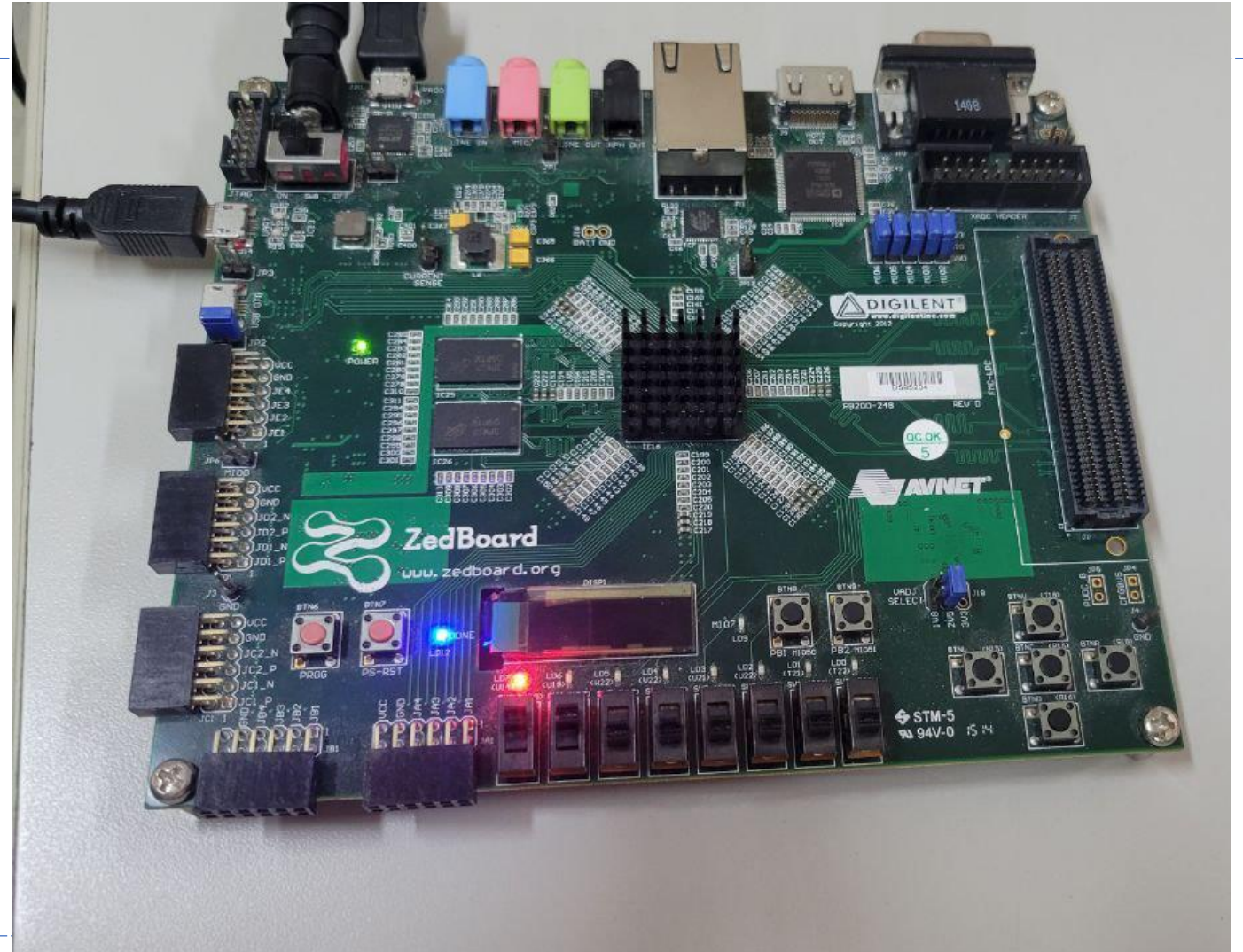
→ Add Bitstream

→ Program



Program to FPGA

4. Check the blue light to confirm successful programming



Outline

- RTL Synthesis and Implementation via Xilinx Vivado
- In-Class Practice
- Software Development via Xilinx SDK
- In-Class Practice



In-class practice

- Adjust the LED blinking frequency based on the switch input of the ZedBoard.

(If there is not enough time, it may become the next homework assignment)



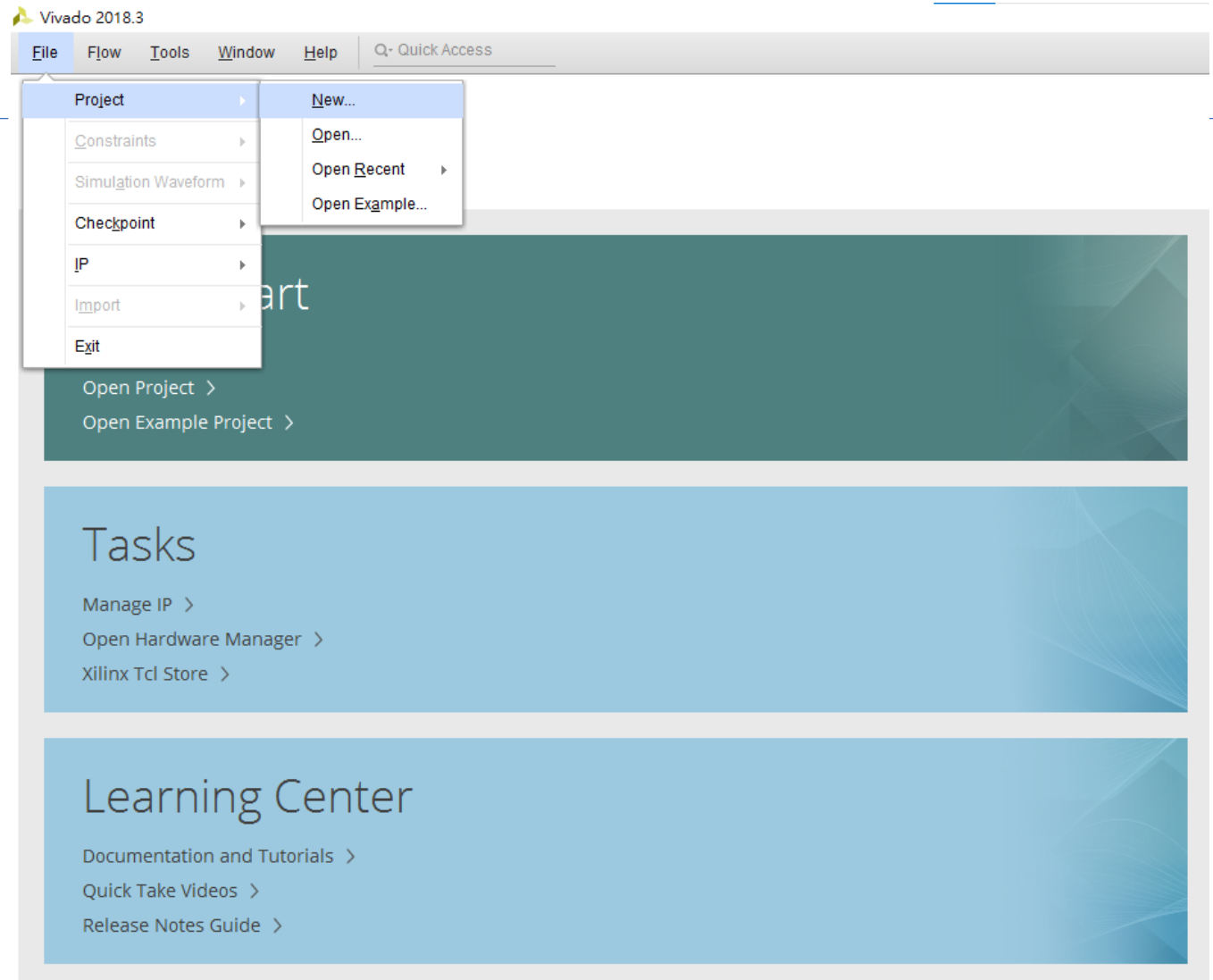
Outline

- RTL Synthesis and Implementation via Xilinx Vivado
- In-Class Practice
- Software Development via Xilinx SDK
 - Create Project
 - Create Block Design
 - SDK Tutorials
- In-Class Practice



Create Project

1. Same as previous step to create a project



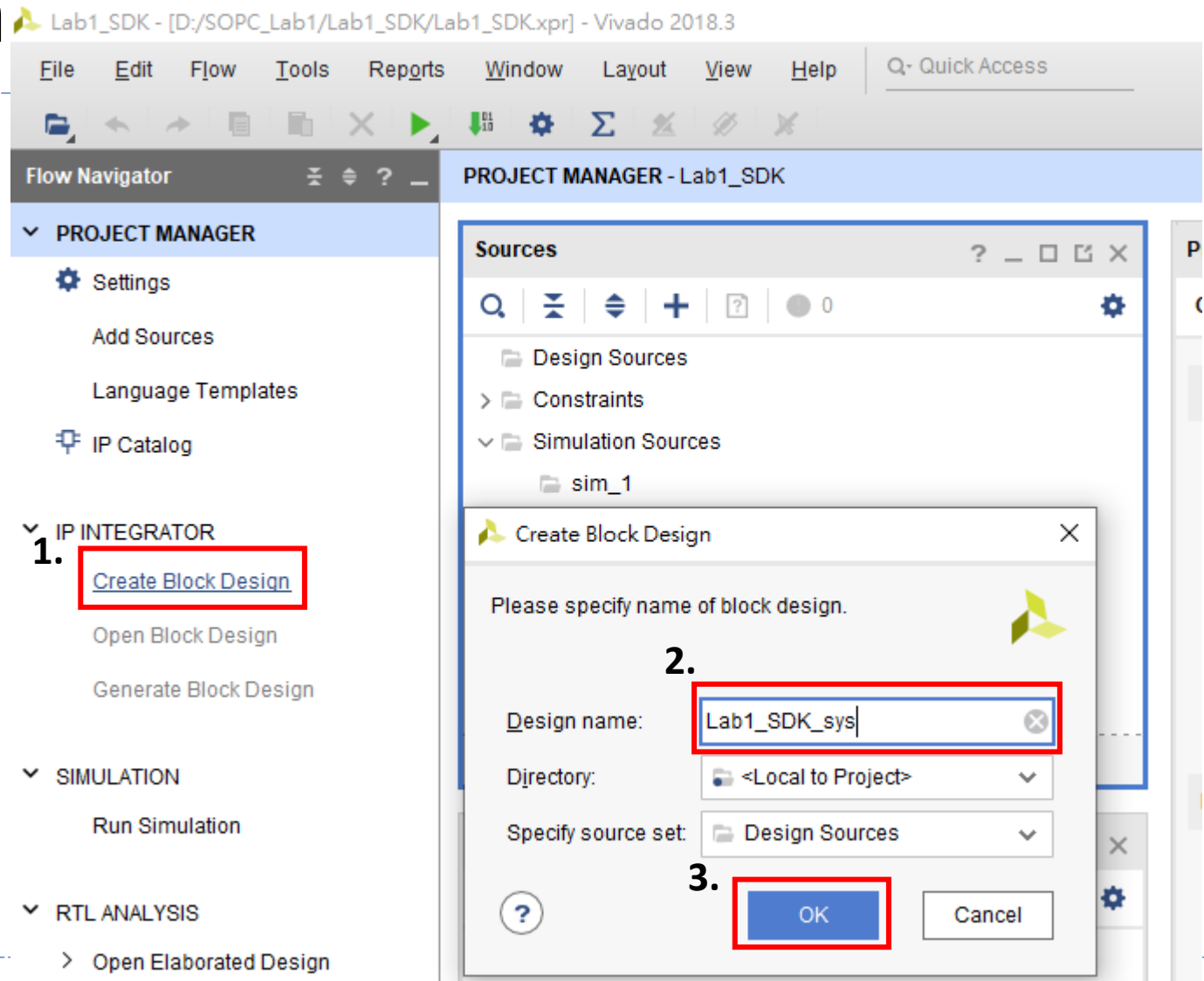
Outline

- RTL Synthesis and Implementation via Xilinx Vivado
- In-Class Practice
- Software Development via Xilinx SDK
 - Create Project
 - Create Block Design
 - SDK Tutorials
- In-Class Practice



Create Block Design

1. Click “ Create Block Design ”
 - Fill “ Design name ”
 - Click “ OK ”

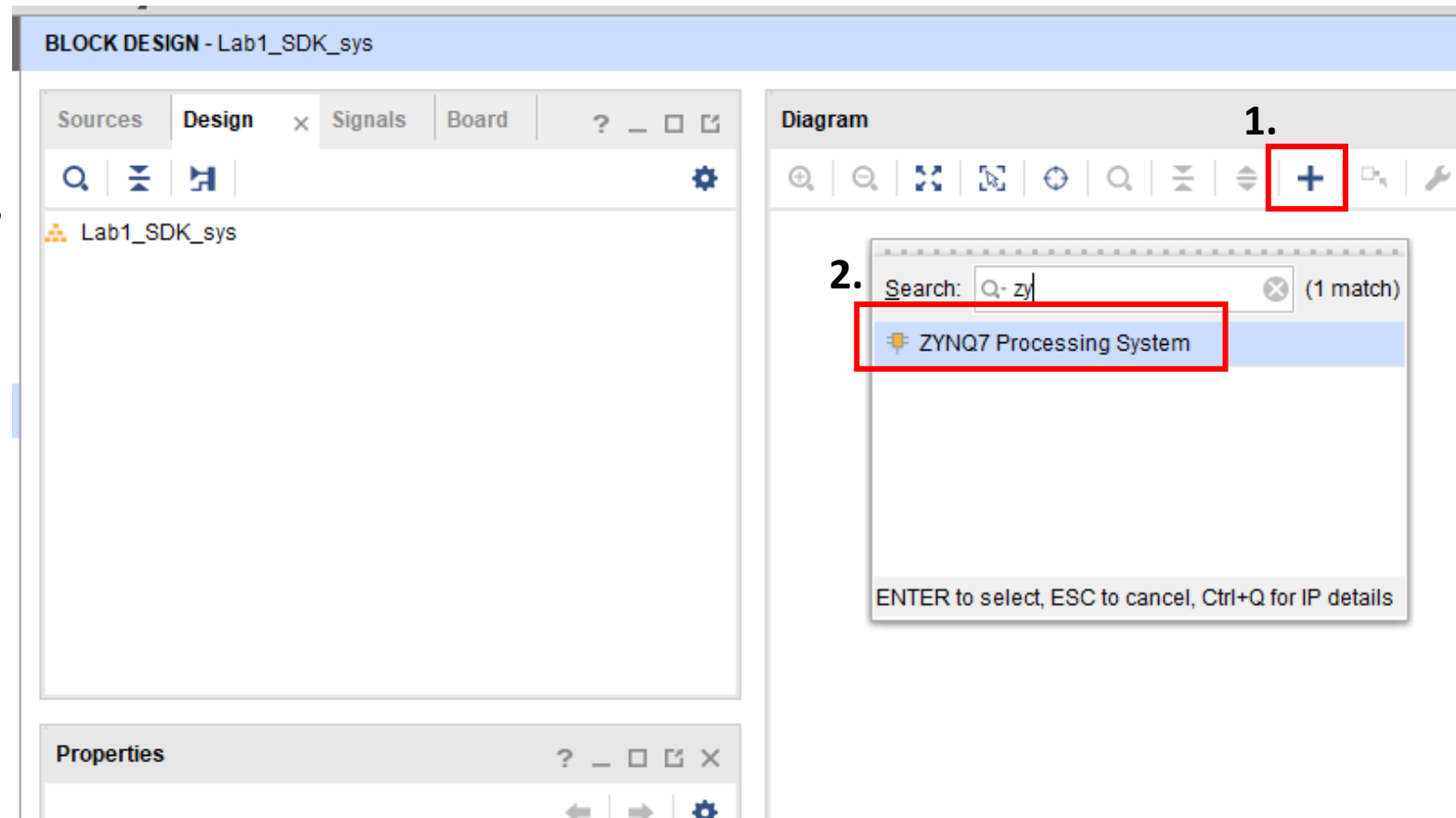


Create Block Design

2. Click “ + ”

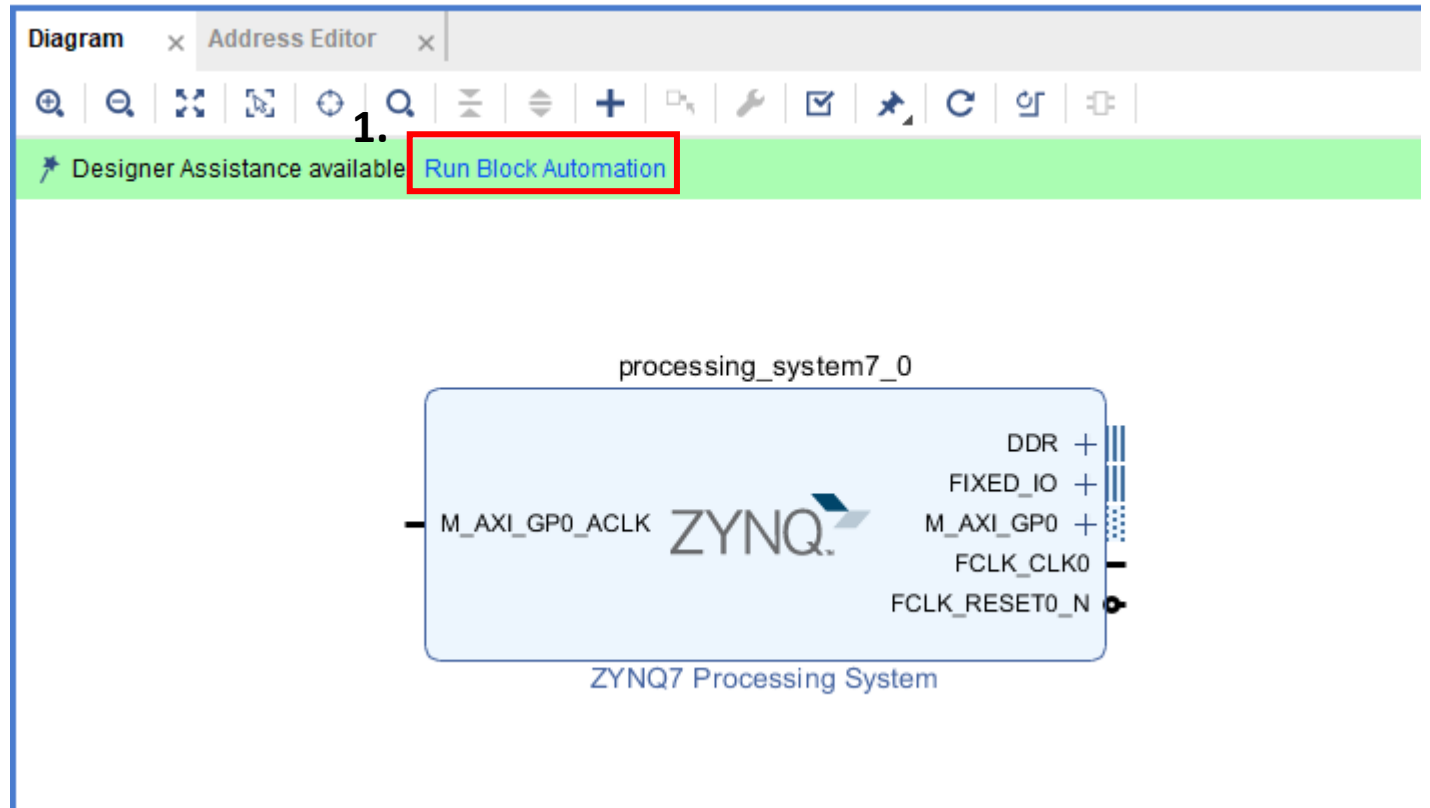
→ Search and Select

“ ZYNQ7 Processing System ”



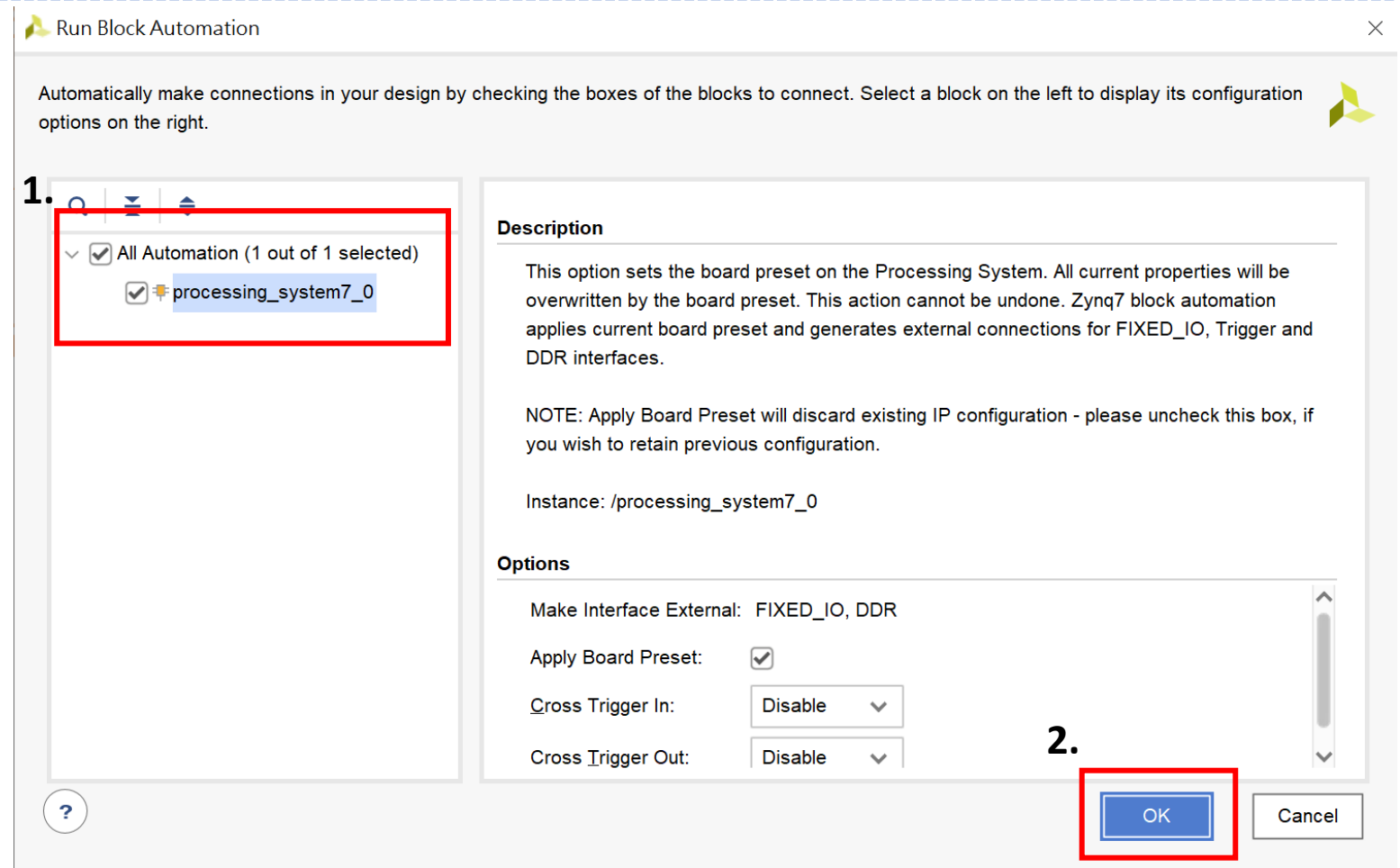
Create Block Design

3. Click “ Run Block Automation ”



Create Block Design

4. Click “ All Automation ”
→ Click “ OK ”



Create Block Design

5. Double Click “Process System”

→ PS-PL Configuration

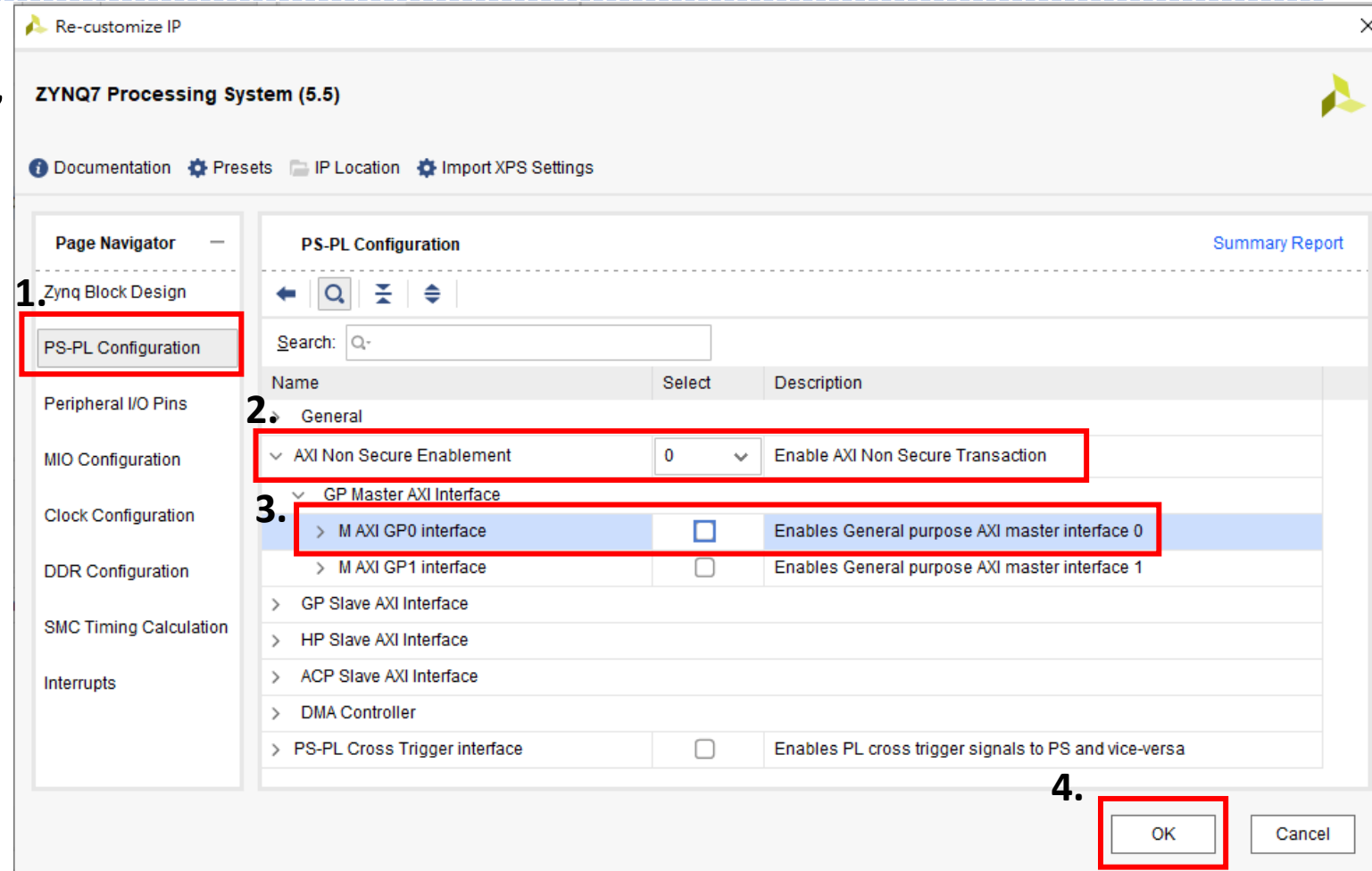
→ AXI Non Secure

→ GP Master AXI Interface

→ Disable the “M_AXI GP0”

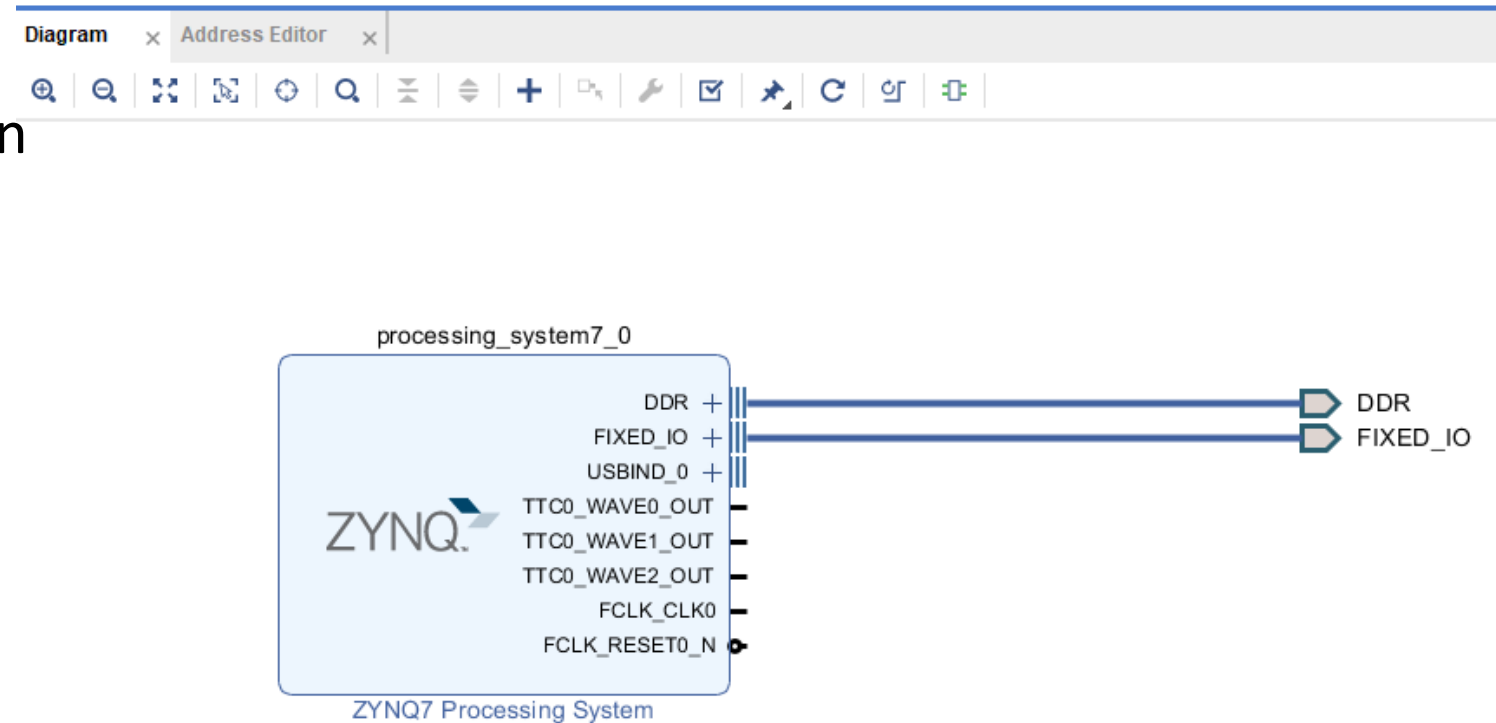
→ Click “OK”

Because in Lab 1, we only use the CPU to run the C code.



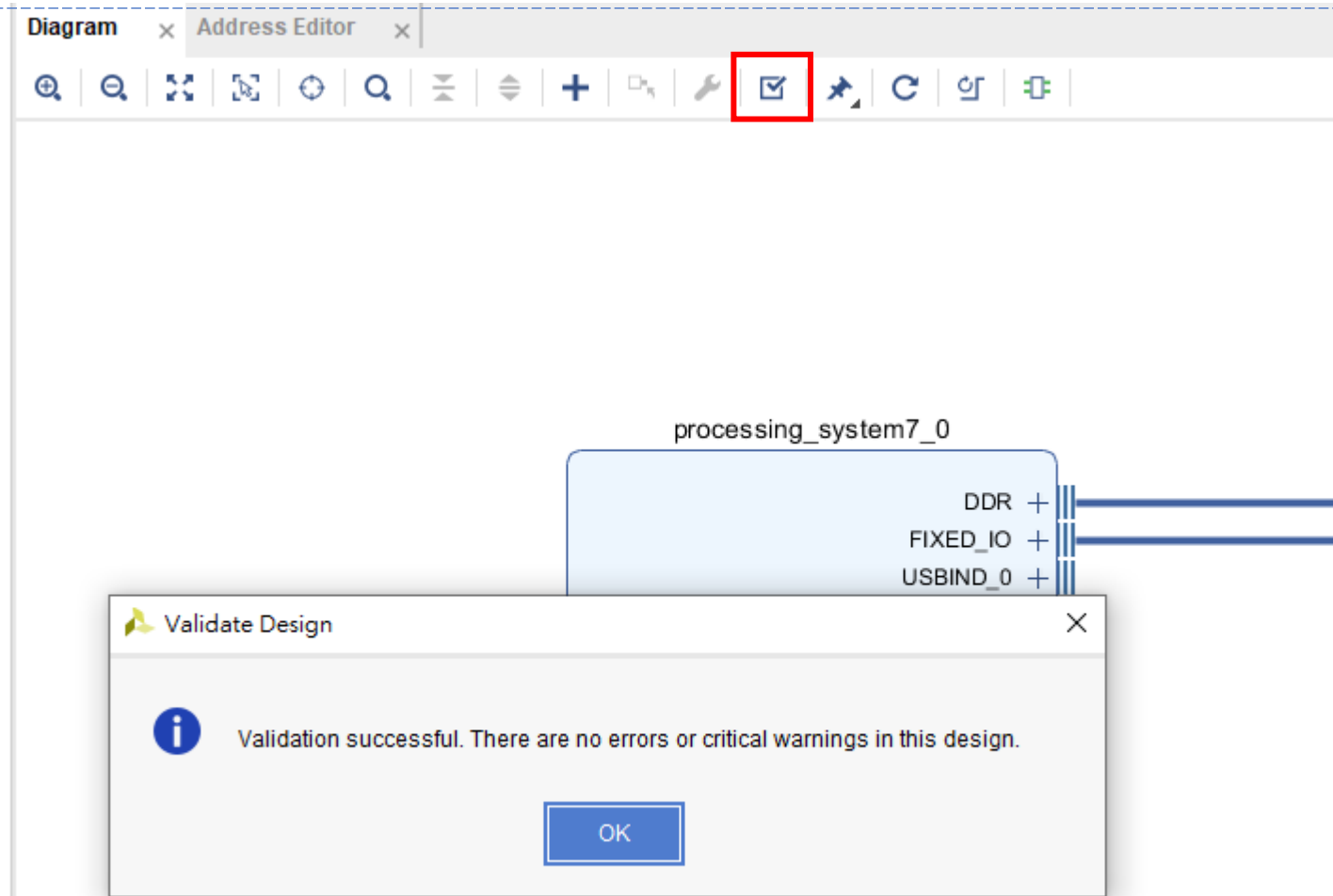
Create Block Design

6. After completing step 5, you can see the AXI port has vanished



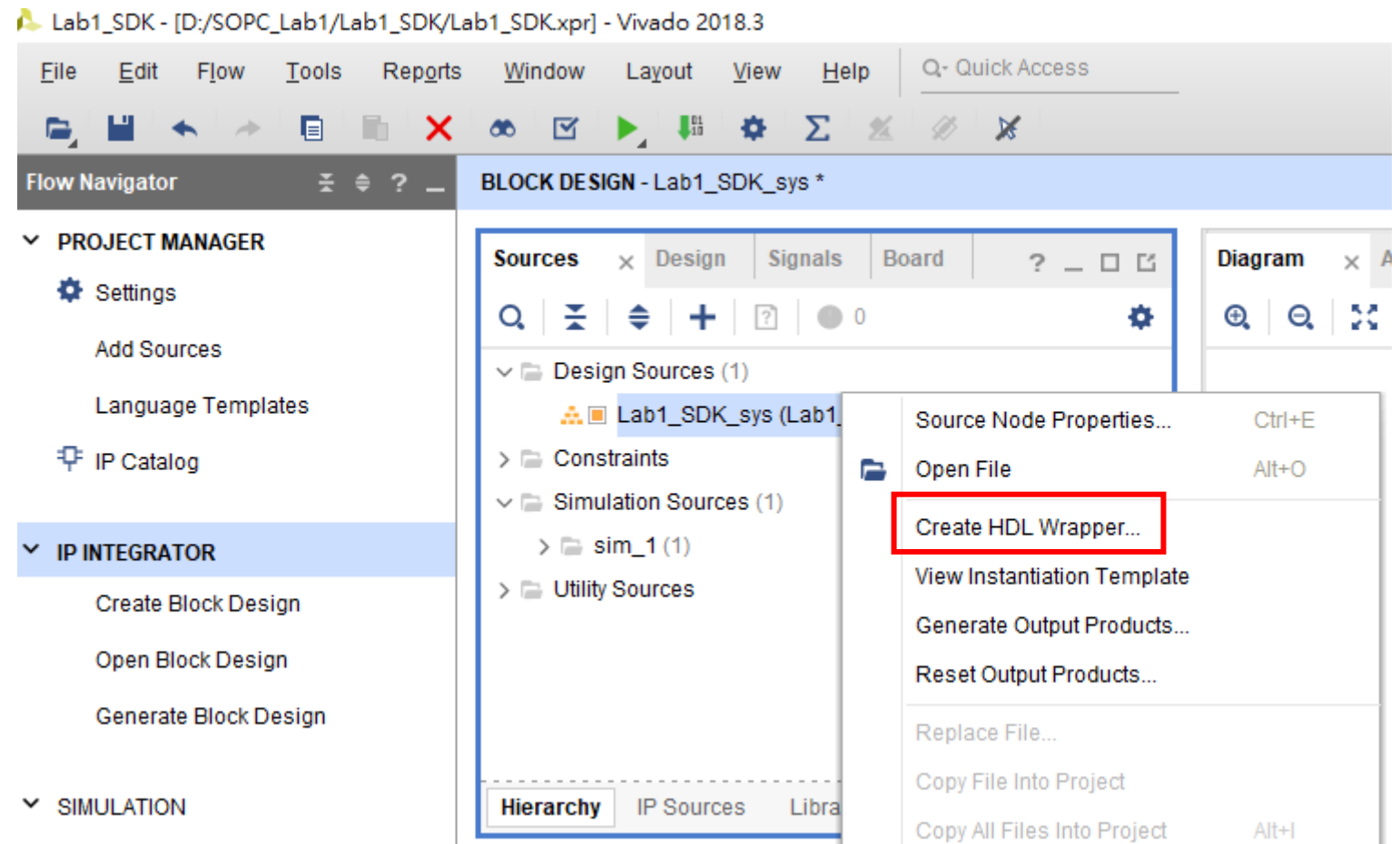
Create Block Design

7. Click “Validate Design”



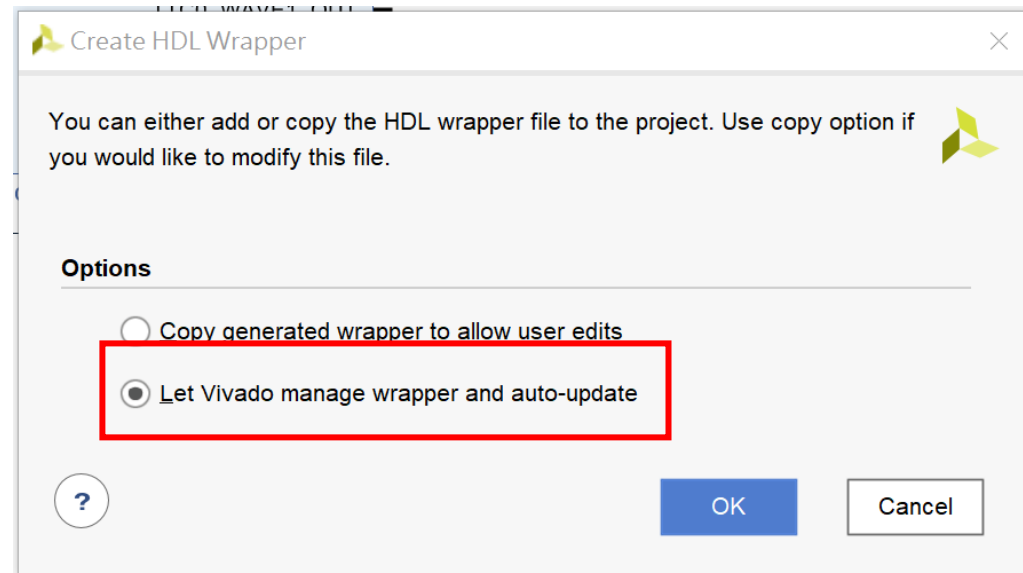
Create Block Design

7. Right Click Block Design
→ Create HDL Wrapper



Create Block Design

8. Let Vivado manage wrapper and auto-update
→ Click “ OK ”



Create Block Design

9. Generate Bitstream

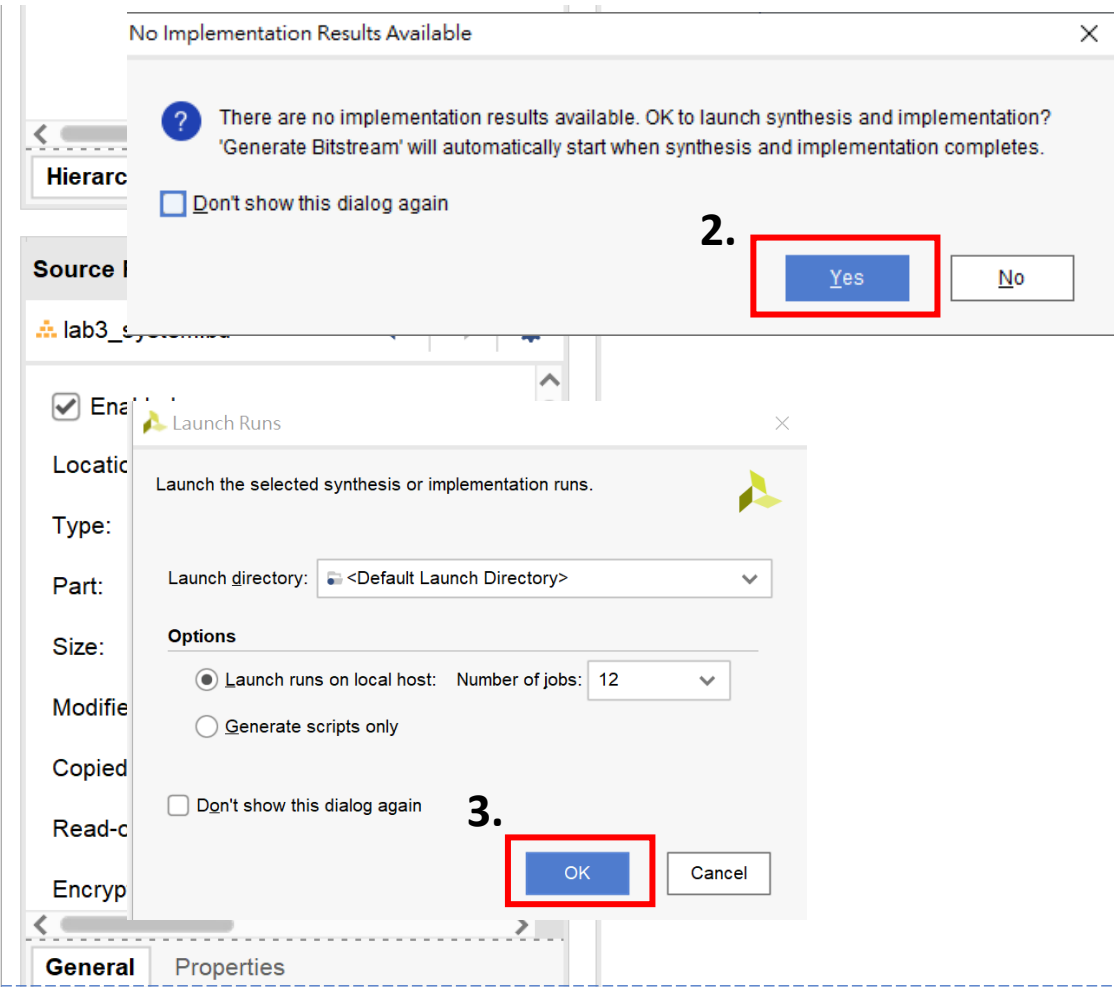
→ Click “ Yes ”

→ Select the Maximum Number of jobs

→ Click “ OK ”

Before generating the bitstream, you have to run **synthesis** and **implementation**

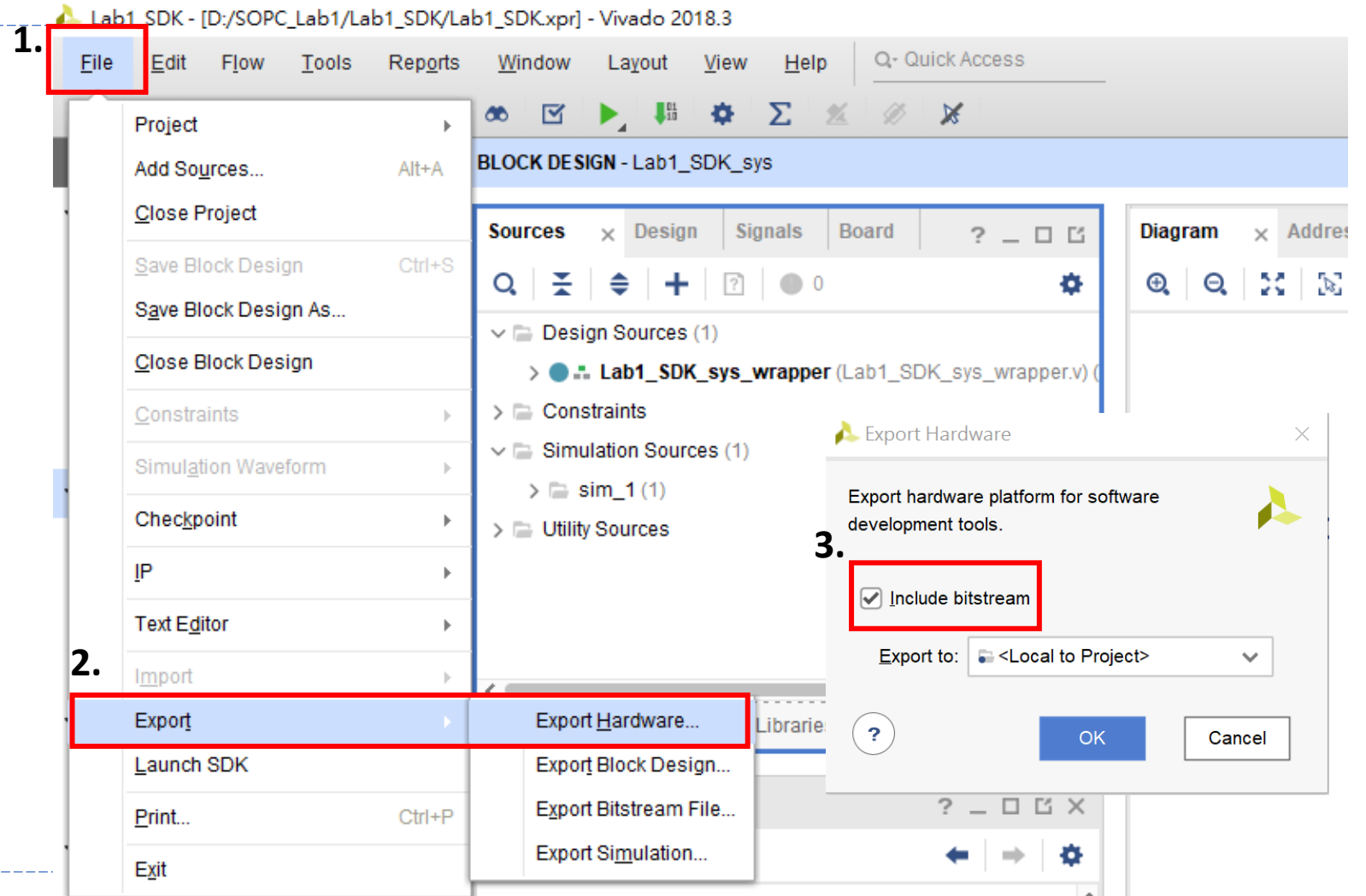
- ▼ RTL ANALYSIS
 - > Open Elaborated Design
- ▼ SYNTHESIS
 - ▶ Run Synthesis
 - > Open Synthesized Design
- ▼ IMPLEMENTATION
 - ▶ Run Implementation
 - > Open Implemented Design
- ▼ PROGRAM AND DEBUG
 - 1. **Generate Bitstream**
 - > Open Hardware Manager



Create Block Design

10. File → Export

- Export hardware
- Include bitstream
- Click “ OK ”



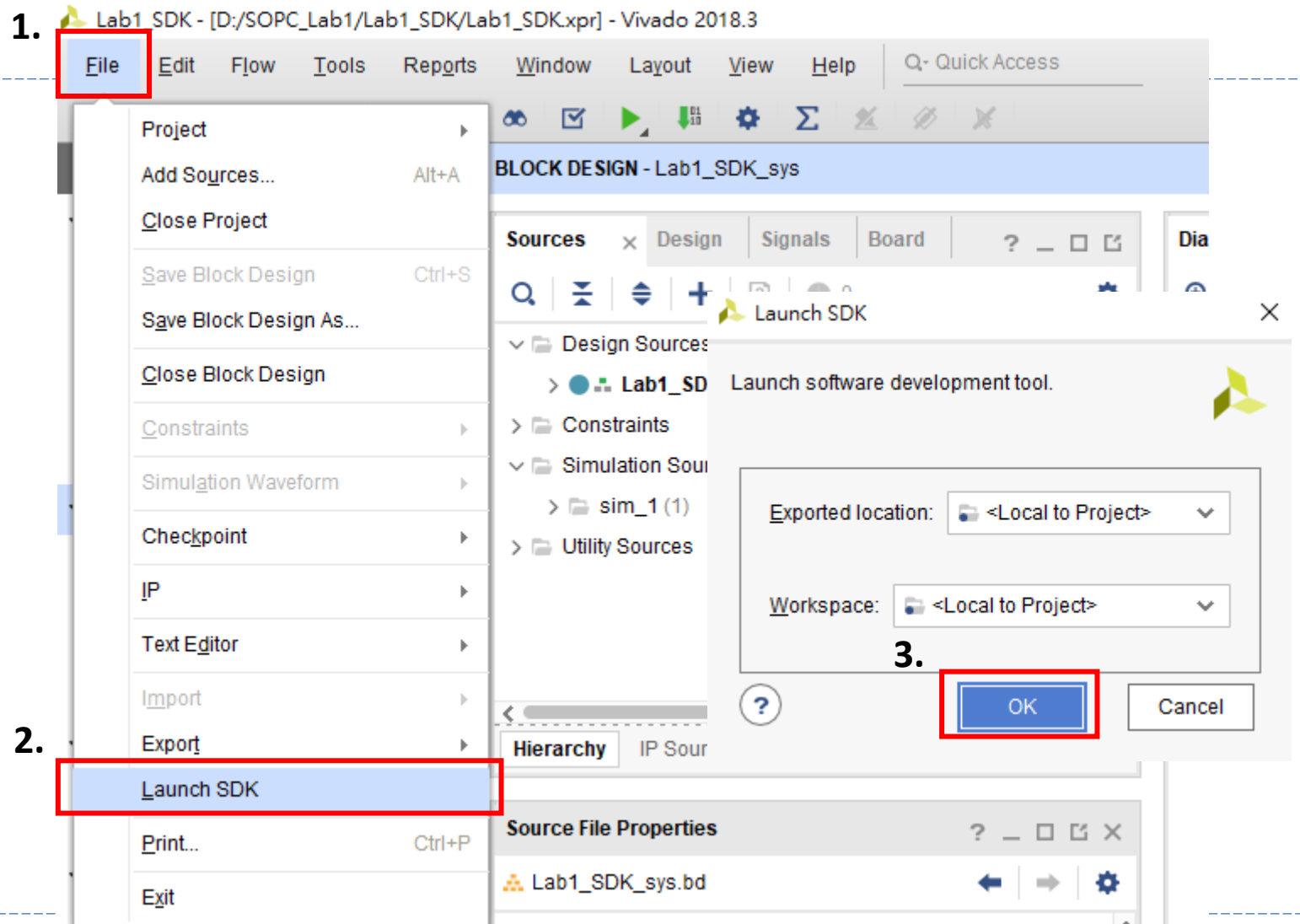
Outline

- RTL Synthesis and Implementation via Xilinx Vivado
- In-Class Practice
- Software Development via Xilinx SDK
 - Create Project
 - Create Block Design
 - SDK Tutorials
- In-Class Practice



SDK Tutorials

1. Click “ File ”
→ Launch SDK
→ OK

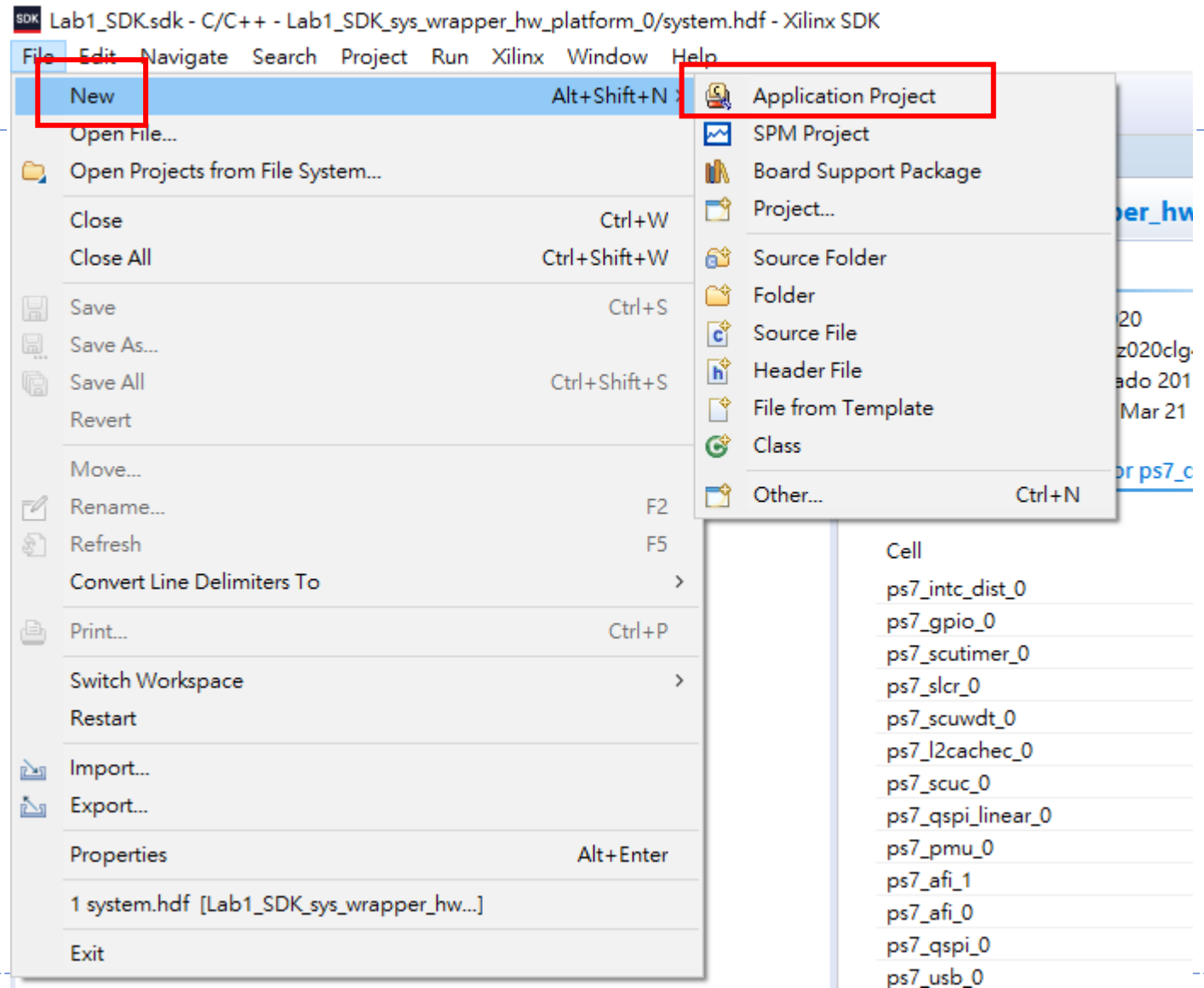


SDK Tutorials

2. Click “ File ”

→ New

→ Application Project



SDK Tutorials

3. Fill Project name
 - Choose the correct “hardware Platform”
 - Click “Next”

The screenshot shows the 'New Project' dialog box in the SDK. The 'Project name' field is set to 'Lab1_SDK'. The 'Hardware Platform' dropdown is set to 'Lab1_SDK_sys_wrapper_hw_platform_0'. The 'Next >' button is highlighted with a red box.

Application Project
Create a managed make application project.

Project name: Lab1_SDK

☒ Use default location
Location: D:\SOPC_Lab1\Lab1_SDK\Lab1_SDK.sdk\Lab1_SDK Browse...

Choose file system: default

OS Platform: standalone

Target Hardware
Hardware Platform: Lab1_SDK_sys_wrapper_hw_platform_0 New...
Processor: ps7_cortexa9_0

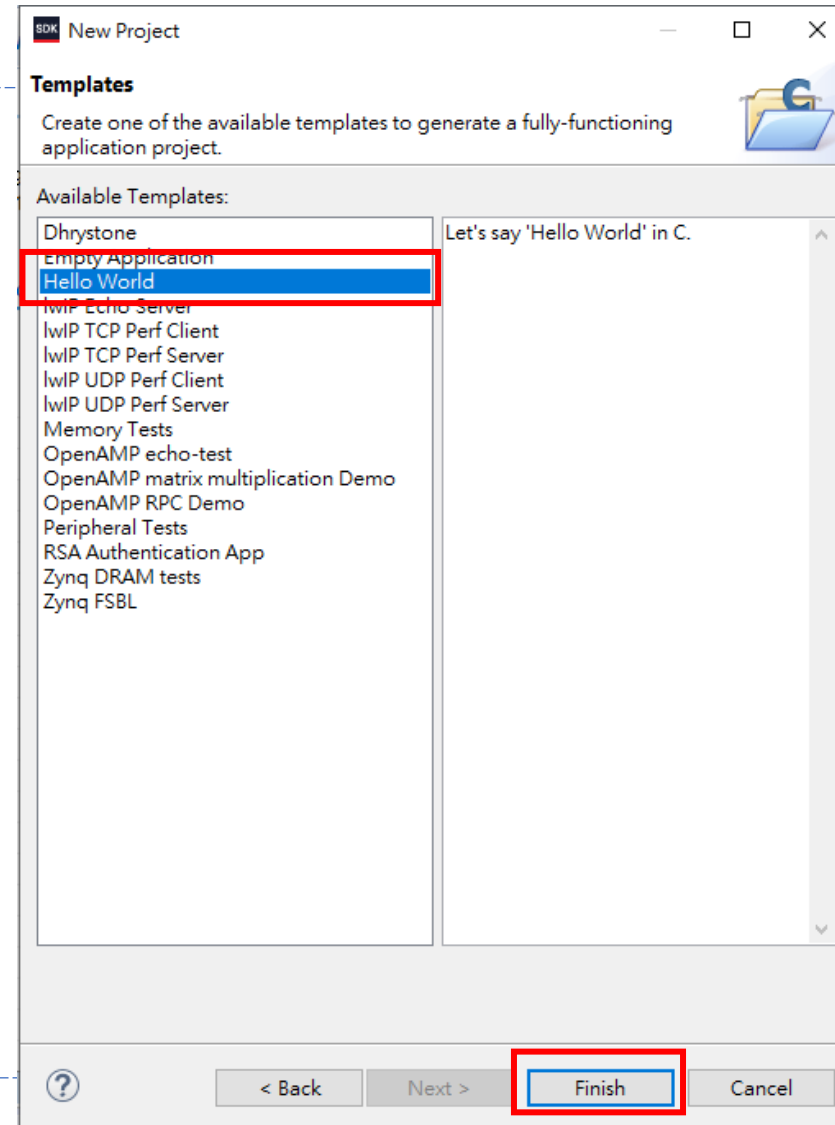
Target Software
Language: ☒ C ☐ C++
Compiler: 32-bit
Hypervisor Guest: N/A
Board Support Package: ☒ Create New Lab1_SDK_bsp
☐ Use existing

< Back **Next >** Finish Cancel

SDK Tutorials

4. Select “ Hello World ”

→ Click “ Finish ”



SDK Tutorials

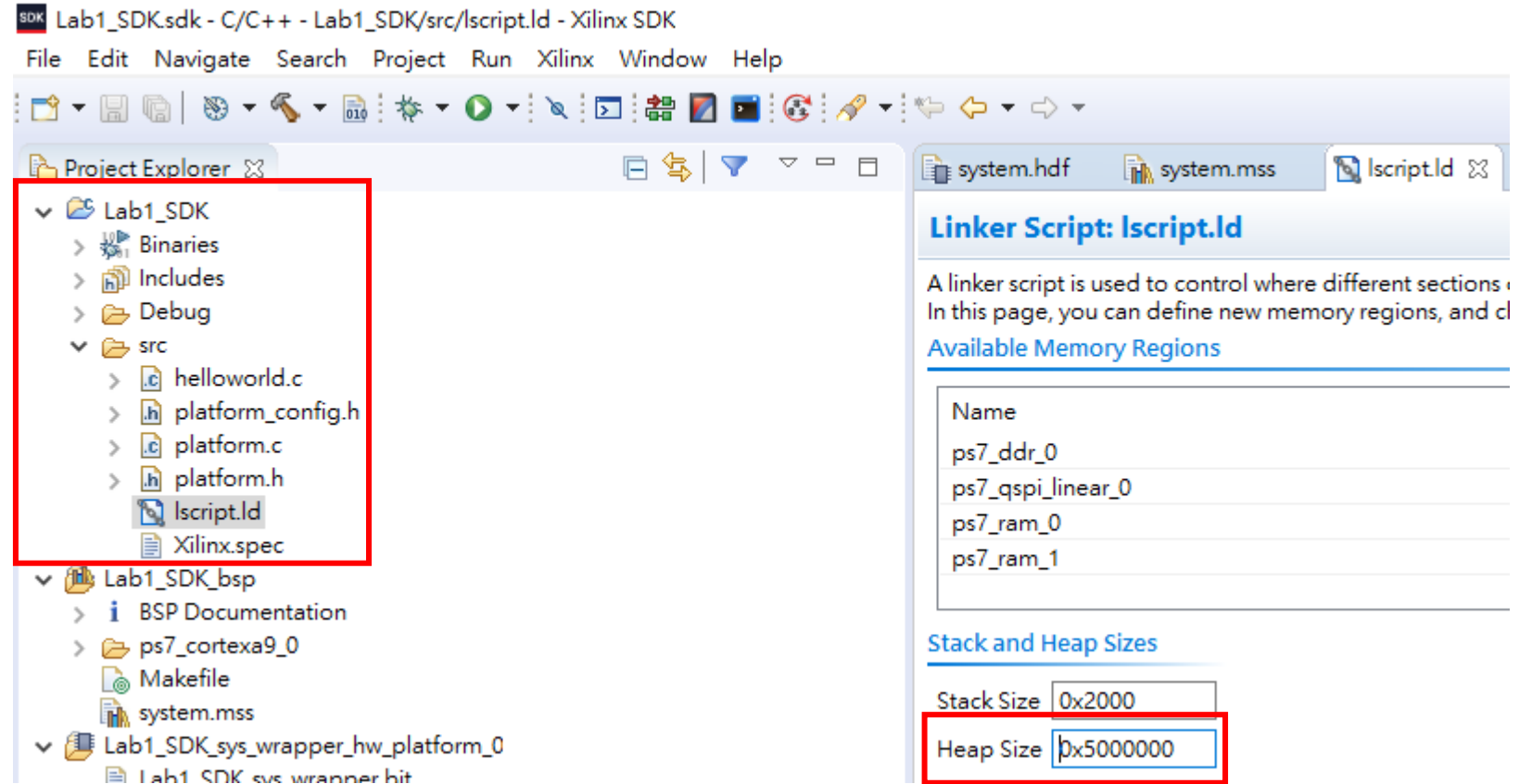
4. Click “ Lab1_SDK ”

→ Click “ Src ”

→ Click “ lscript.ld ”

→ adjust “ heap space ”

Because run.c uses many **malloc** functions, it requires a lot of **heap space**

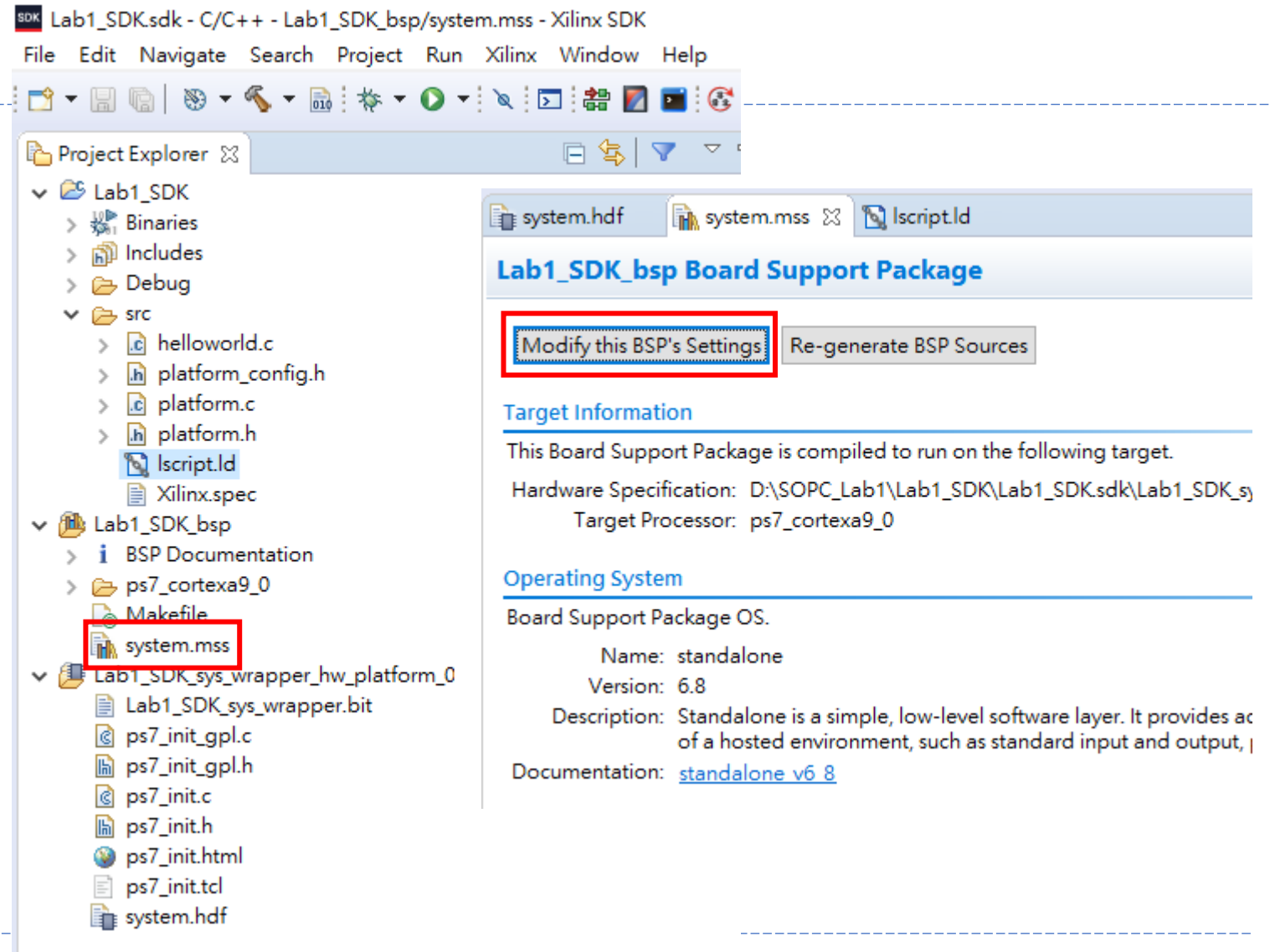


SDK Tutorials

5. How to set up “xilffs”

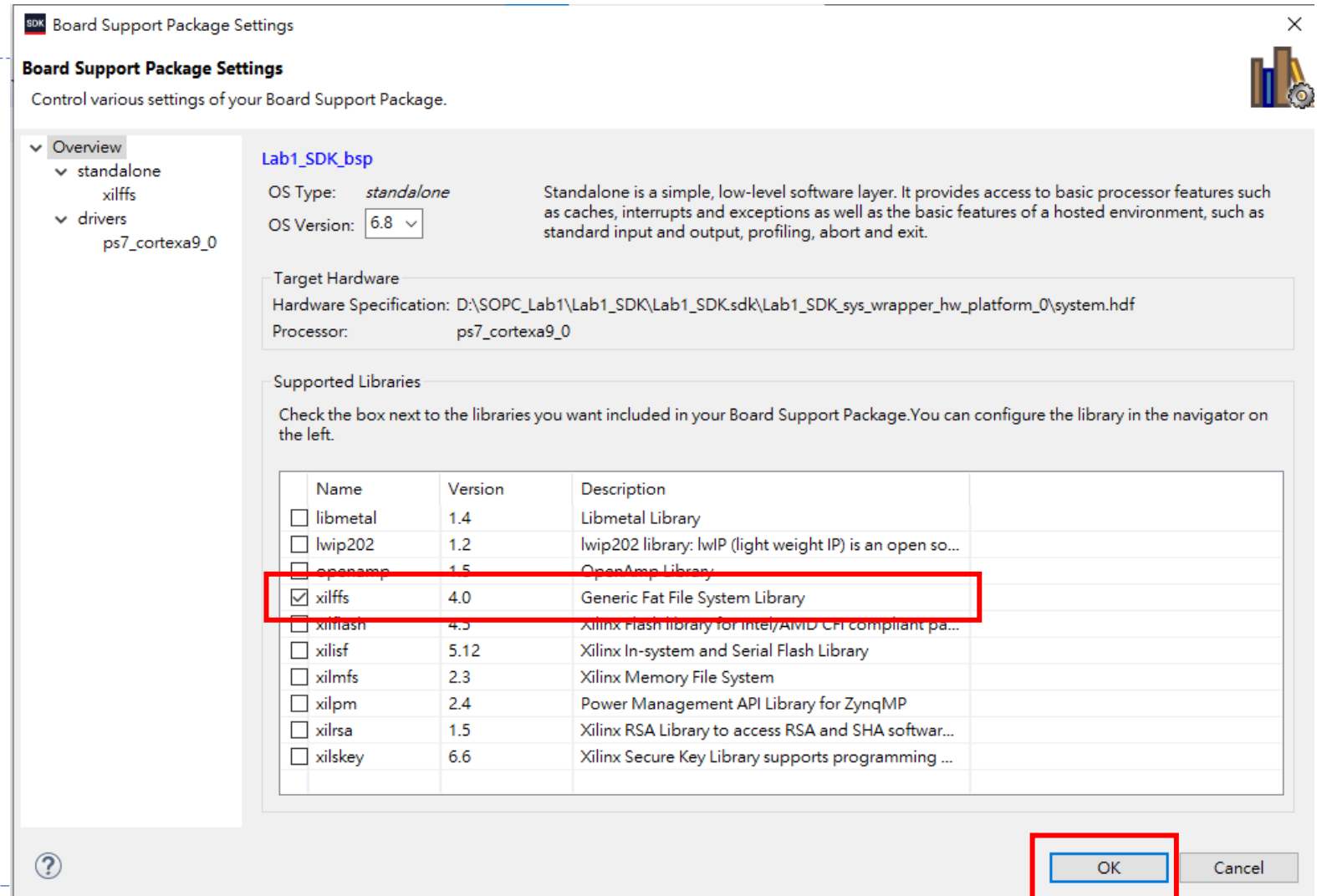
- Click “ Lab1_SDK_bsp ”
- Click “ system.mss ”
- Click “ Modify the BSP ”

SDK does not support the way
run.c reads files



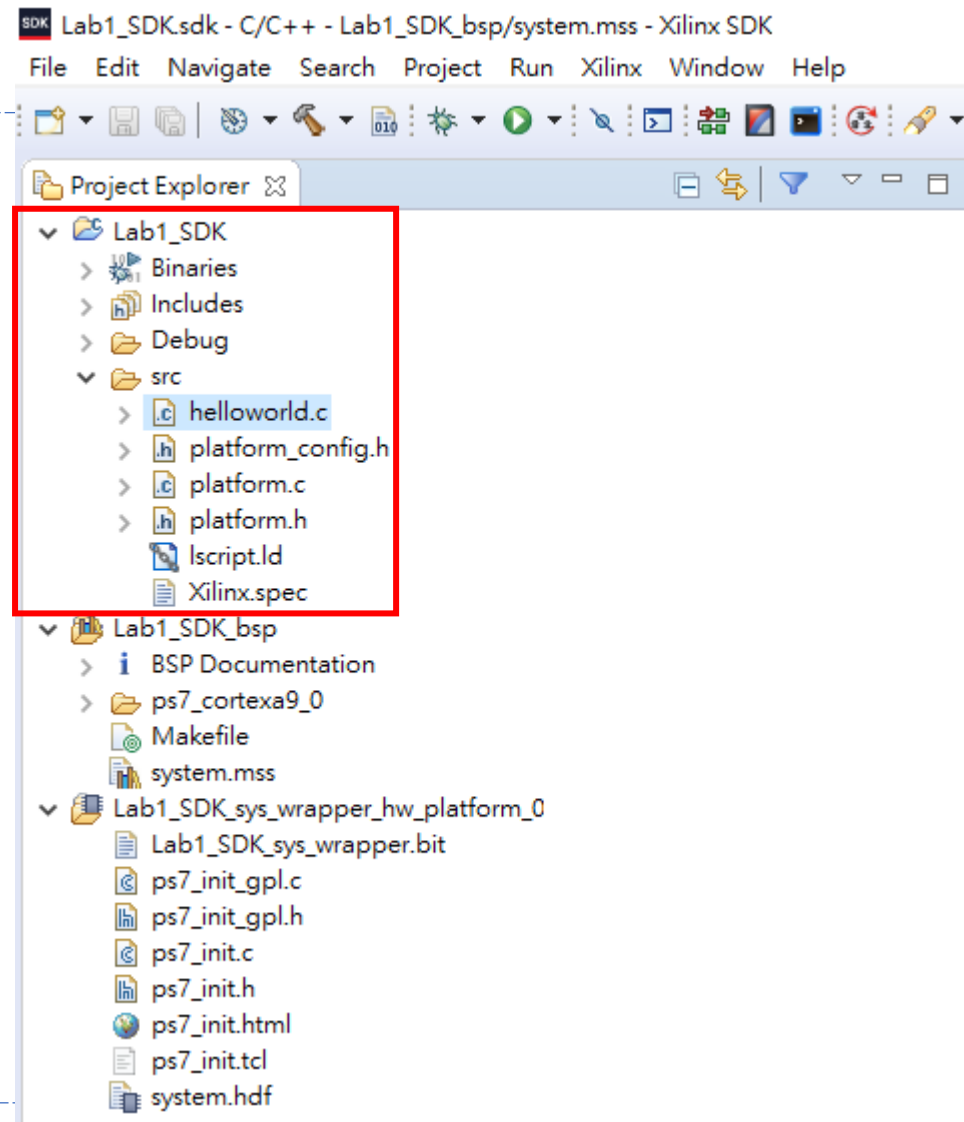
SDK Tutorials

6. Click “xilffs”
→ Click “OK”



SDK Tutorials

7. Click “ Project1_SDK ”
 - Click “ Src ”
 - Click “ helloworld.c ”
(Edit C code here)



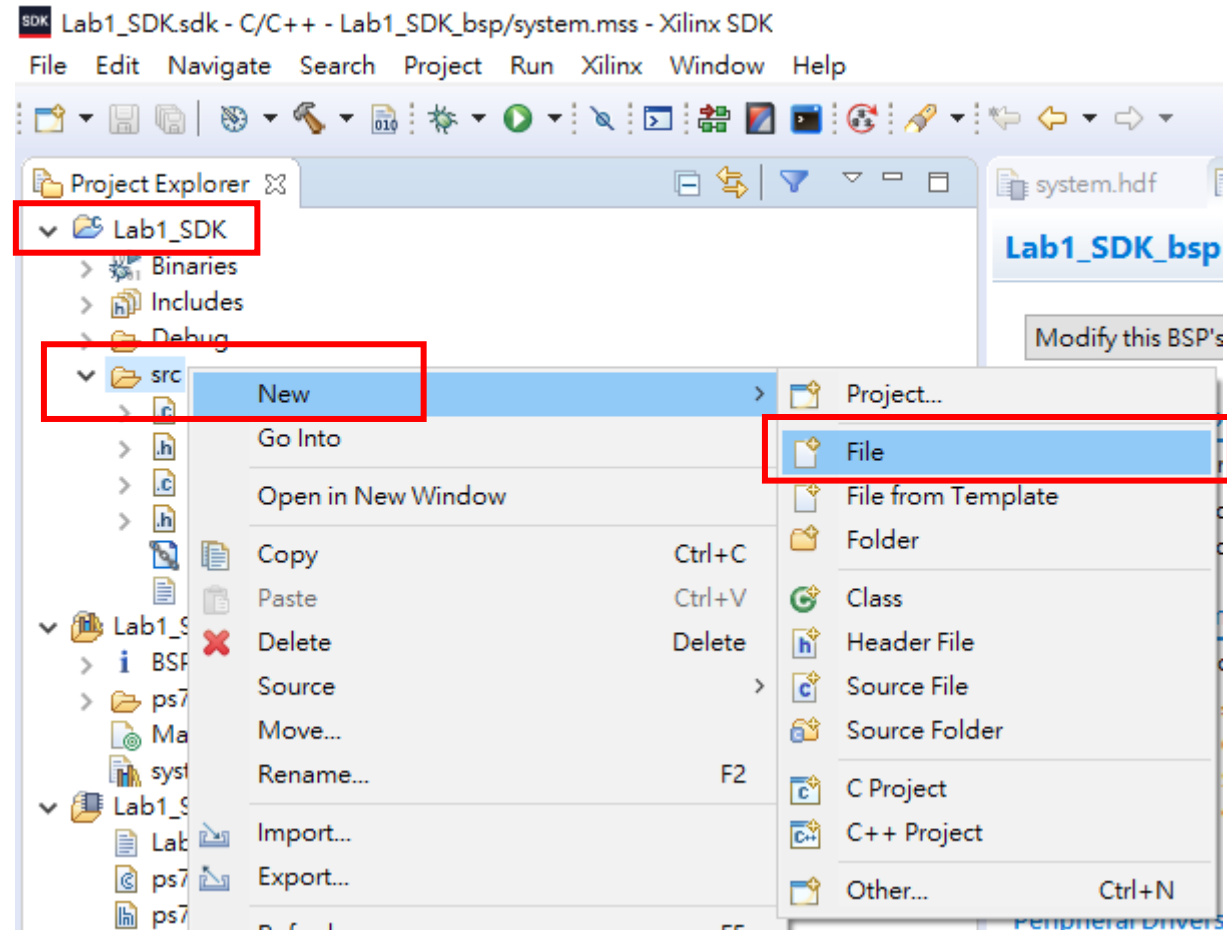
SDK Tutorials

8. Click “ Lab1_SDK ”

→ Right Click “ Src ”

→ Click “ New ”

→ Click “ File ”



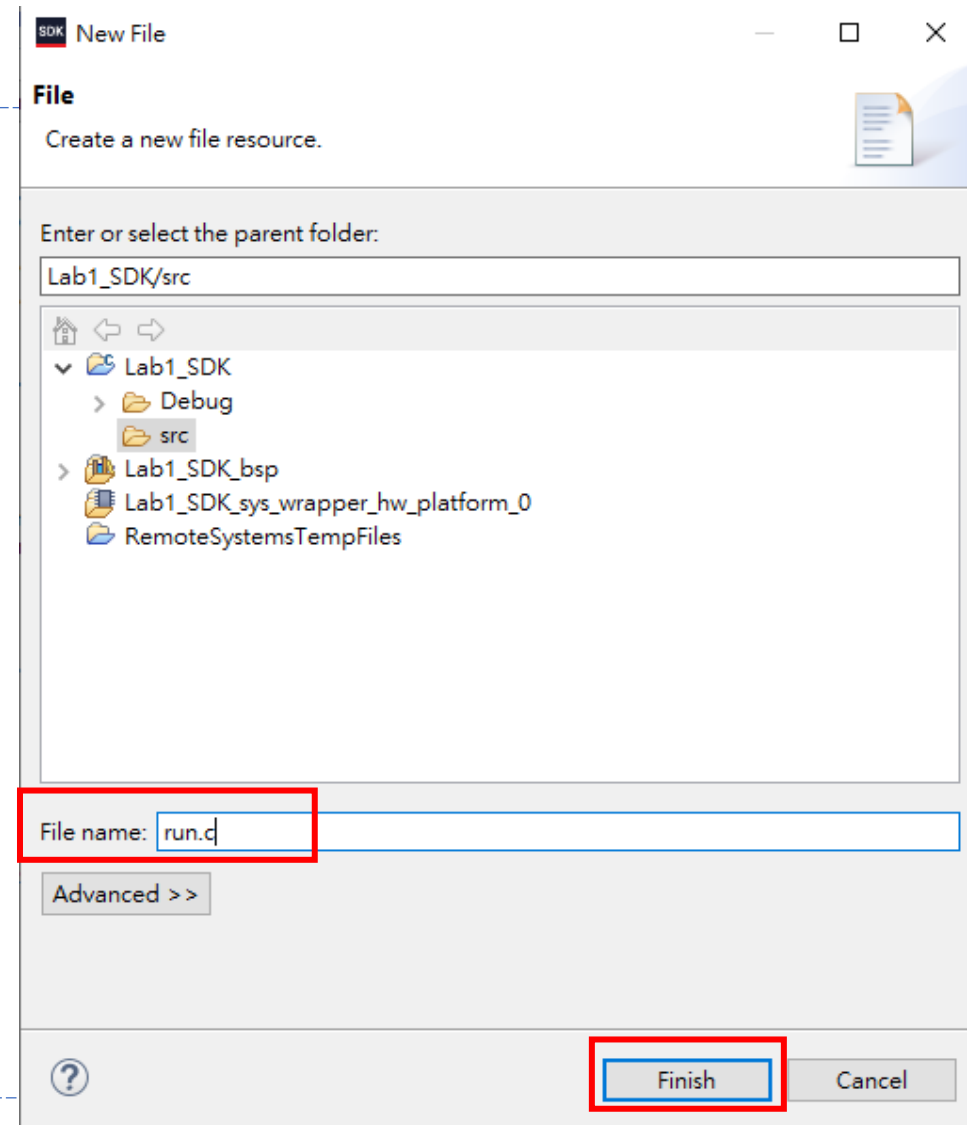
SDK Tutorials

9. Fill “File name”

→ Finish

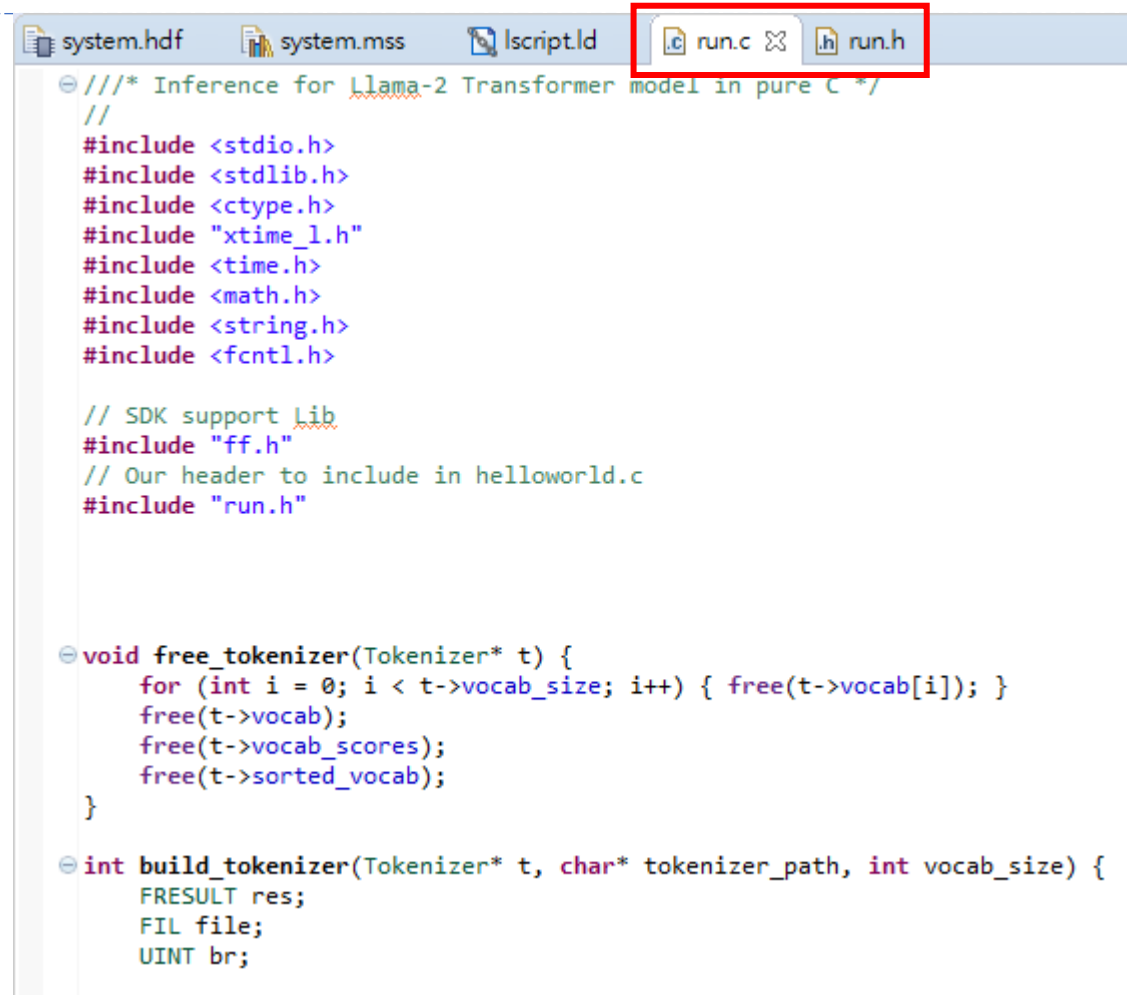
In this step, we add the required files (run.c, run.h) to the project

You can also use “Advance”, to directly link the download files



SDK Tutorials

10. Copy the code we provided to your SDK
(helloworld.c , run.c , run.h)



```
system.hdf system.mss lscript.ld run.c run.h
/** Inference for Llama-2 Transformer model in pure C */
//
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include "xtime_1.h"
#include <time.h>
#include <math.h>
#include <string.h>
#include <fcntl.h>

// SDK support lib
#include "ff.h"
// Our header to include in helloworld.c
#include "run.h"

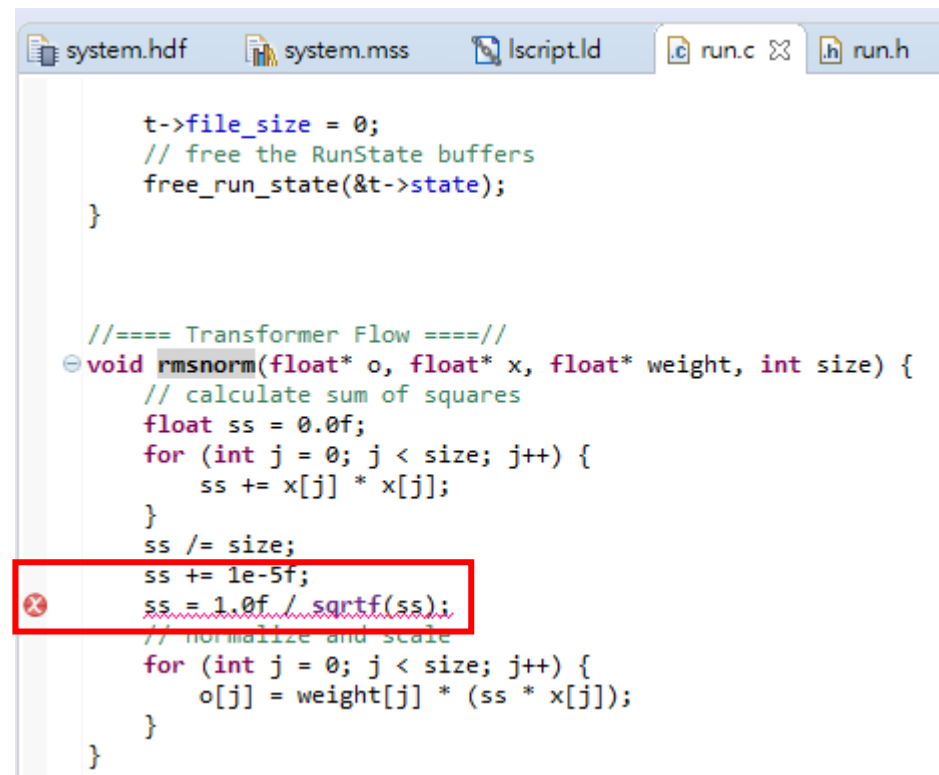
void free_tokenizer(Tokenizer* t) {
    for (int i = 0; i < t->vocab_size; i++) { free(t->vocab[i]); }
    free(t->vocab);
    free(t->vocab_scores);
    free(t->sorted_vocab);
}

int build_tokenizer(Tokenizer* t, char* tokenizer_path, int vocab_size) {
    FRESULT res;
    FIL file;
    UINT br;
```

SDK Tutorials

11. How to resolve the library linking error

Although we have included `<math.h>` in `run.c`, `run.c` is a newly added file, so SDK cannot automatically link to the math library.



The screenshot shows a code editor with several tabs: `system.hdf`, `system.mss`, `lscript.ld`, `run.c`, and `run.h`. The `run.c` tab is active, displaying the following C code:

```
t->file_size = 0;
// free the RunState buffers
free_run_state(&t->state);
}

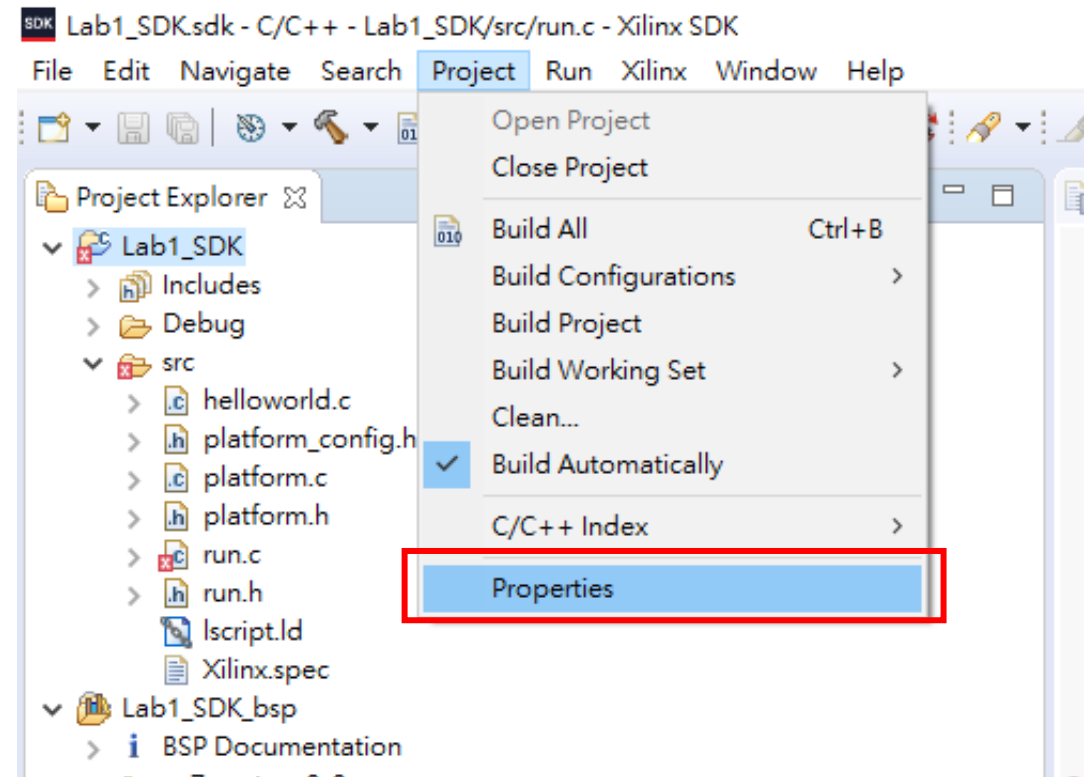
//=== Transformer Flow ===//
void rmsnorm(float* o, float* x, float* weight, int size) {
    // calculate sum of squares
    float ss = 0.0f;
    for (int j = 0; j < size; j++) {
        ss += x[j] * x[j];
    }
    ss /= size;
    ss += 1e-5f;
    ss = 1.0f / sqrtf(ss);
    // normalize and scale
    for (int j = 0; j < size; j++) {
        o[j] = weight[j] * (ss * x[j]);
    }
}
```

A red rectangle highlights the line `ss = 1.0f / sqrtf(ss);`, which has a red 'x' icon to its left, indicating a linking error. The error occurs because the `sqrtf` function is not linked to the `math` library.

SDK Tutorials

12. How to re-link Library

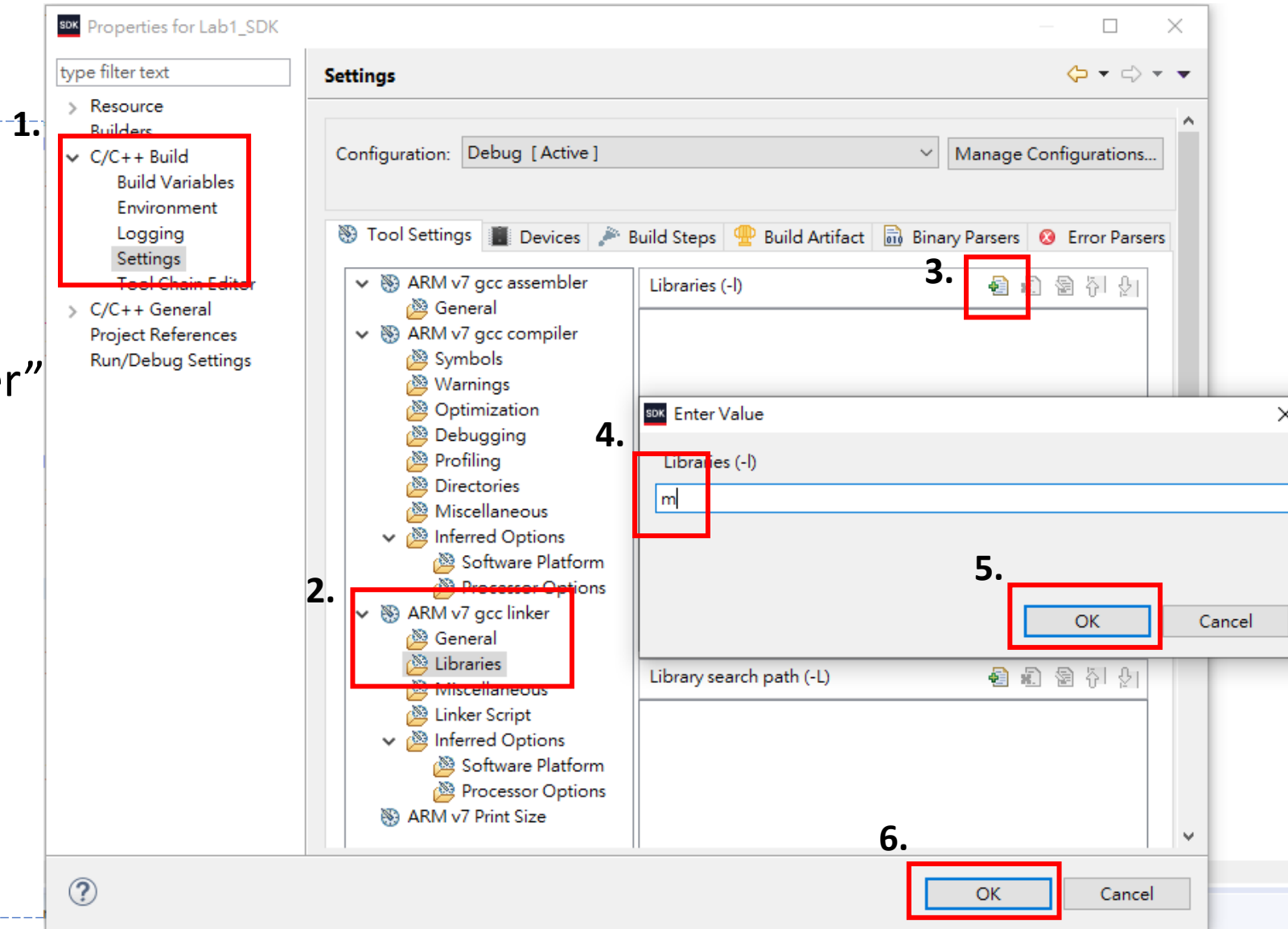
- Click “ Project ”
- Click “ Properties ”



SDK Tutorials

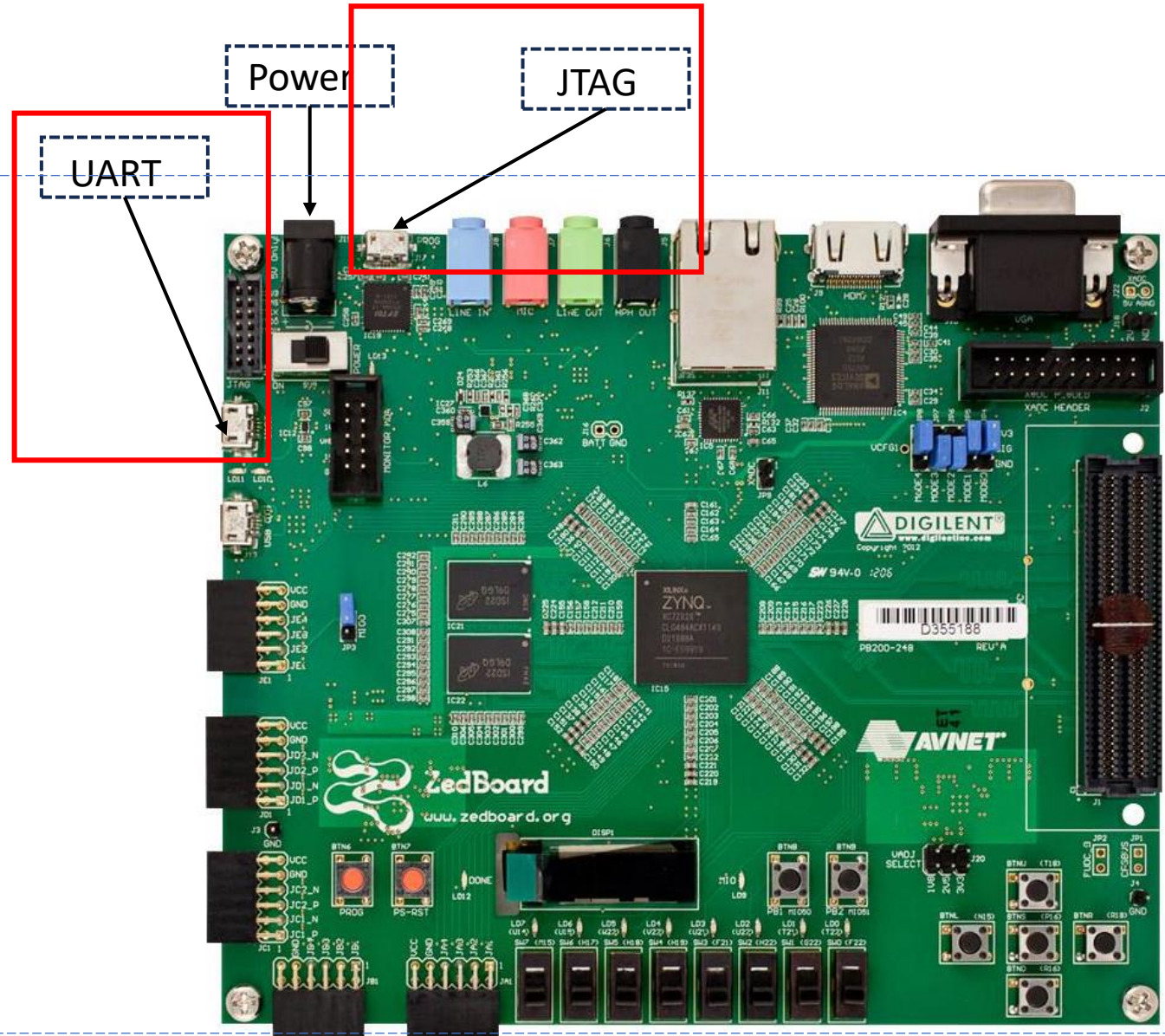
13. How to re-link Library

- Click “ C++ Build ”
- Click “ Settings ”
- Click “ ARM v7 gcc linker ”
- Enter “ m ”
- Click “ OK ”



SDK Tutorials

14. Connect ZedBoard with Power , UART , JTAG cable



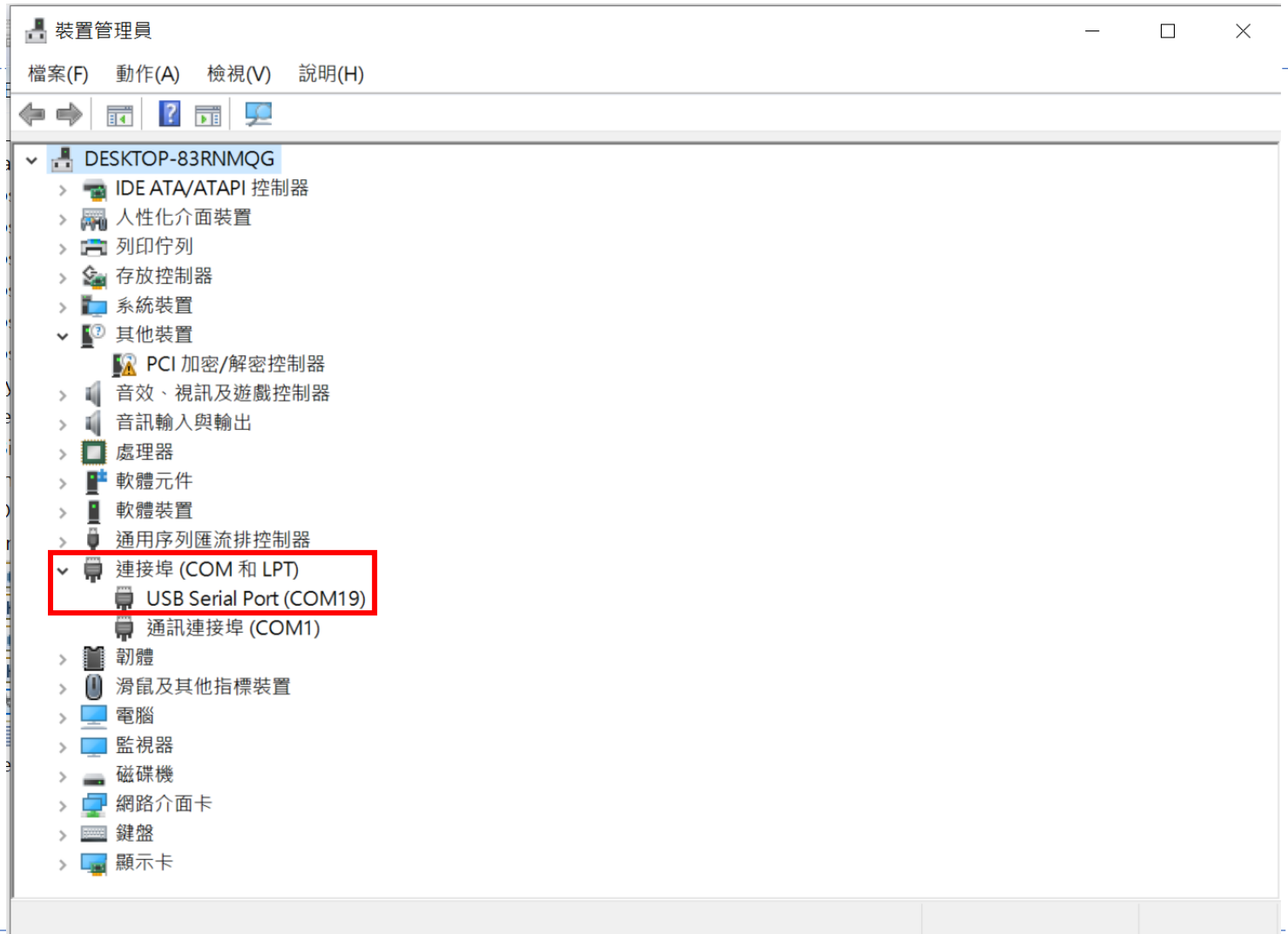
SDK Tutorials

15. Open ZedBoard Power

→ Device manager

→ Find the COM port

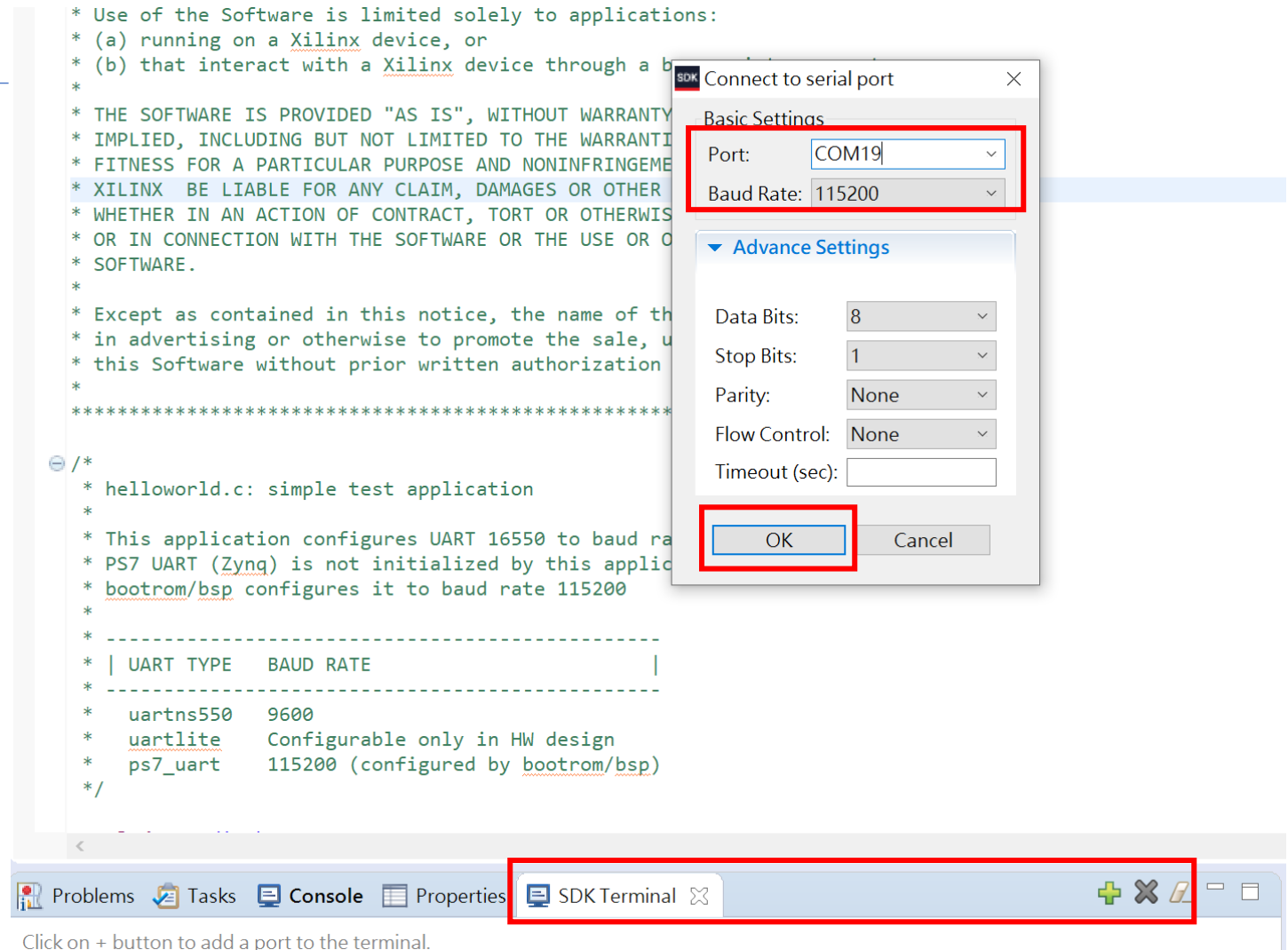
Your PC might not have the same
COM port as pictures



SDK Tutorials

16. SDK Terminal

- Click  Button
- Select the COM port
- OK

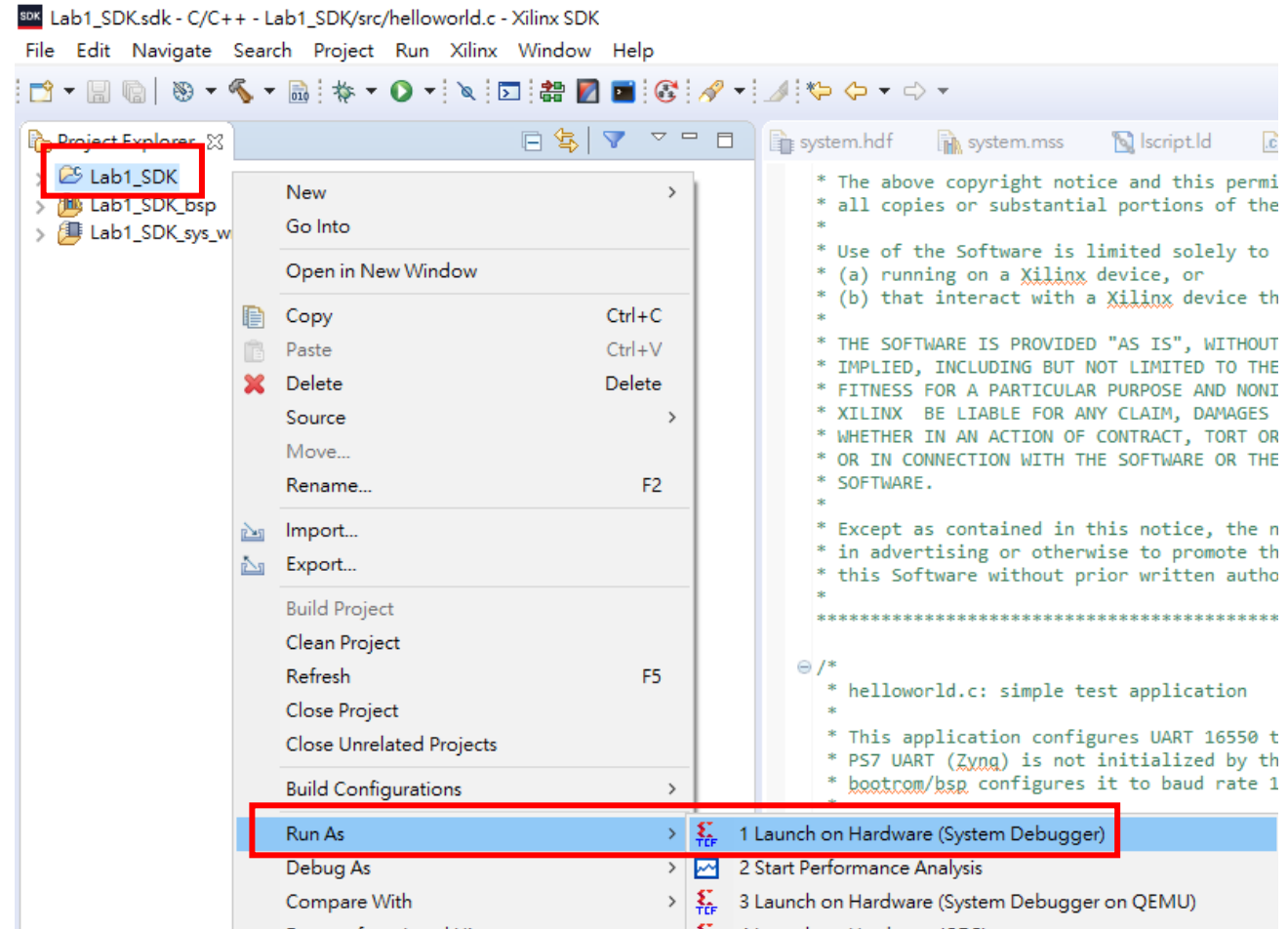


Click on + button to add a port to the terminal.

SDK Tutorials

17. Right Click “ Lab1_SDK ”

- Run as
- Launch on Hardware (system debugger)



SDK Tutorials

18. Execution Result

You can try different **prompts**
and **step counts**

```
===== GENERATION OUTPUT =====
```

```
Prompt: One day, Lily
```

```
One day, Lily and Ben went to the park with their mom. They saw a big slide, a swing, and a sandbox. Lily wanted to try them all. She ran to the slide and climbed up the ladder. Ben followed her.
```

```
"Be careful, Lily!" Ben said. "The slide is very high and fast. You might fall and hurt yourself."
```

```
Lily nodded. She climbed up the ladder and sat on the slide. She looked down and saw how high it was. She felt scared. She wanted to go back, but she was too afraid.
```

```
"Ben, help me!" she cried. "I don't want to slide down. It's too high and fast."
```

```
Ben ran to her and helped her get off the slide. He hugged her and said, "Don't worry, Lily. I'll help you. You can do it. I'm here for you."
```

```
Lily smiled and said, "Thank you, Ben. You're a good friend. I'm sorry I was scared. I was silly."
```

```
Ben smiled back and said, "It's okay, Lily. I'm glad
```

```
achieved tok/s: 1.307464
```

```
===== GENERATION DONE =====
```



Outline

- RTL Synthesis and Implementation via Xilinx Vivado
- In-Class Practice
- Software Development via Xilinx SDK
- In-Class Practice



In-class practice

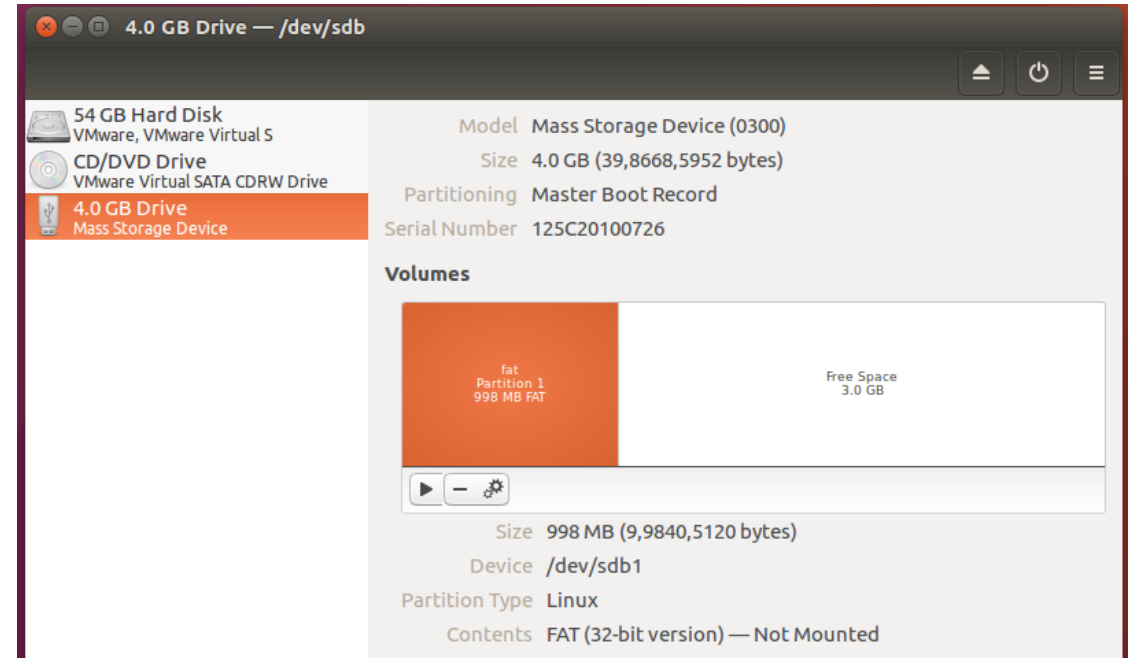
- Please replace “ GVM ” in your homework with “ matmul ”
in run.c
- Modify runq.c by referring to the run.c file we provided.
(If there is not enough time, it may become the next homework assignment)



Additional Explanation

1. How to store .bin

- Use VMware to partition the SD card into a 1 GB FAT partition

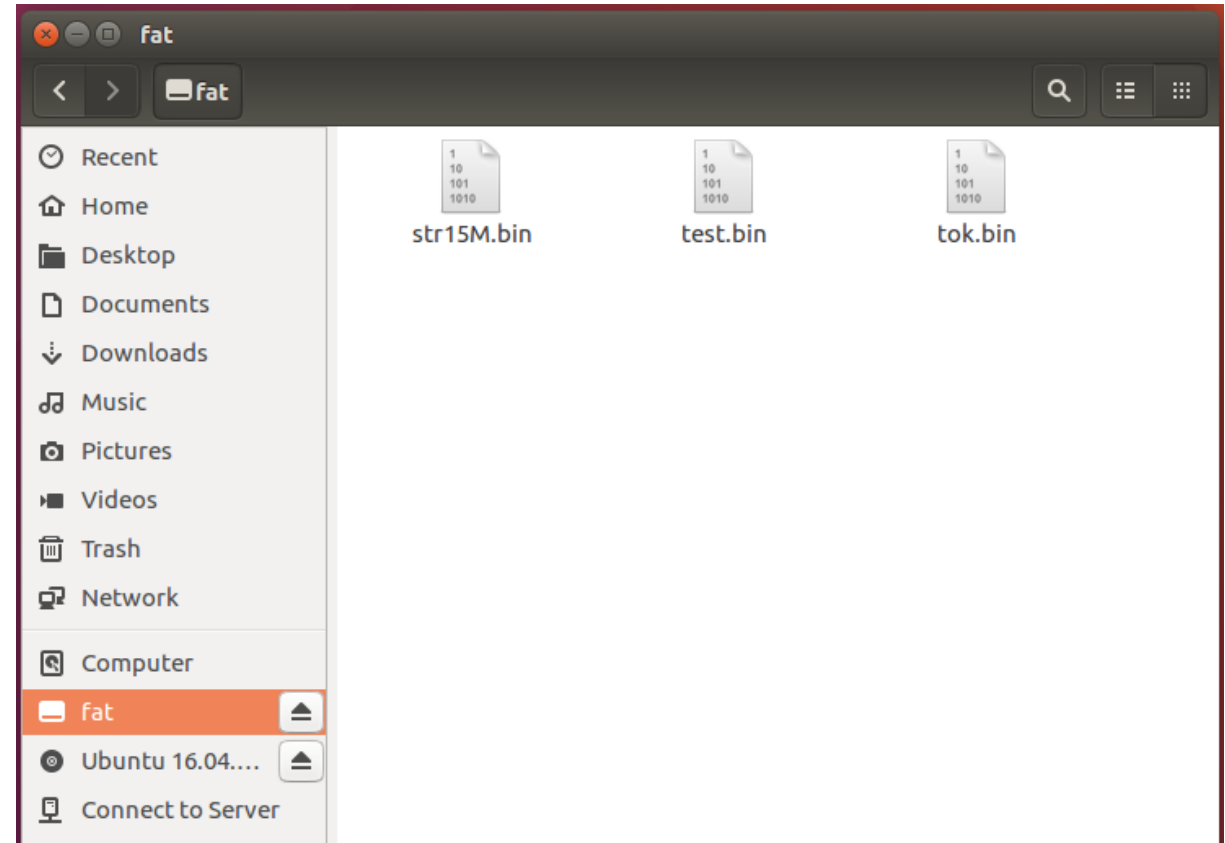


Additional Explanation

1. How to store .bin

- Copy tokenizer.bin and stories15M.bin to the SD card

f_open has a filename length limit



Additional Explanation

2. How to modify run.c

- Because .bin stores in SD card, we have to mount the device initially.

```
/** f_mount(arg_1, arg_2, arg_3)
/** arg_2: mount the first device
/** arg_3: 0 means delay mount (until use can check whether mount is successful)
//          1 means immediate mount (can check whether mount is successful immediately)

res = f_mount(&fatfs, "0:/", 1);
printf("f_mount res = %d\r\n", res);
if (res != FR_OK) {
    cleanup_platform();
    return 0;
}
```



Additional Explanation

2. How to modify run.c

- SDK doesn't support "fopen", so we have to use its "xilffs" api

Equivalent functions:

1. fopen -> f_open
2. fread -> f_read
3. fseek -> f_lseek
4. fclose -> f_close

```
/** f_open(arg_1, arg_2, arg_3)
/** arg_1: record which file is used
/** arg_2: open which file
/** arg_3: tell FatFS the file only read

res = f_open(&file, checkpoint, FA_READ);
if (res != FR_OK) {
    printf("Couldn't open file %s, res=%d\r\n", checkpoint, res);
    return -1;
}
```

```
/** r_read(arg_1, arg_2, arg_3, arg_4)
/** arg_1: which file
/** arg_2: store data read from
/** arg_3: read how many byte
/** arg_4: how many byte actually read

// read config header
res = f_read(&file, config, sizeof(Config), &br);
if (res != FR_OK || br != sizeof(Config)) {
    printf("failed to read config, res=%d, br=%u\r\n", res, br);
    f_close(&file);
    return -2;
}
```

Additional Explanation

2. How to modify run.c

- SDK doesn't support "mmap", so we have to use "malloc" and "f_read".

```
*data = mmap(NULL, *file_size, PROT_READ, MAP_PRIVATE, *fd, 0);
```

➤ Original run.c

```
// allocate full model buffer
*data = (unsigned char*)malloc(*file_size);
if (*data == NULL) {
    printf("malloc model buffer failed, size=%u\r\n", *file_size);
    f_close(&file);
    return -4;
}

// read whole file into DDR buffer
res = f_read(&file, *data, *file_size, &br);
if (res != FR_OK || br != *file_size) {
    printf("f_read whole model failed, res=%d, br=%u\r\n", res, br);
    free(*data);
    *data = NULL;
    f_close(&file);
    return -5;
}
```

➤ modified run.c



Additional Explanation

2. How to modify run.c

- SDK doesn't support "clock_gettime", so we have to use "Xtime"

```
long time_in_ms() {  
    // return time in milliseconds, for benchmarking the model speed  
    struct timespec time;  
    clock_gettime(CLOCK_REALTIME, &time);  
    return time.tv_sec * 1000 + time.tv_nsec / 1000000;  
}
```

➤ Original run.c

```
//=== utilities: time ===//  
long long time_in_ms() {  
    XTime t;  
    XTime_GetTime(&t);  
    return (long long)(t * 1000 / COUNTS_PER_SECOND);  
}
```

➤ modified run.c

